

AXI4-AVIP

Contents

Chapter 1 INTRODUCTION	7
Chapter 2 ARCHITECTURE	22
Chapter 3 IMPLEMENTATION	24
3.1 Pin Interface	24
3.2 Testbench Components	26
Chapter 4 Directory Structure	42
4.1 Package Content	42
Chapter 5 Configuration	44
5.1 Global Package Variables	44
Chapter 6 Verification Plan	52
6.1 Verification Plan	52
Chapter 7 Assertion Plan	60
Chapter 8 Coverage	63
Chapter 9 Test Cases	70
9.1 Test Flow / Transaction	70
9.4 Sequences / Virtual Sequences / Test Cases / Testlists	82
Chapter 10 User Guide	85
Chapter 11 References	86

List of Tables

Table — Name of the table	Pg. no
1.1 Write Address Signals	11
1.2 Write Data Signals	12
1.3 Write Response Signals	12
1.4 Read Address Signals	13
1.5 Read Data Signals	13
3.1 AXI4 pins used to interface to external devices	24
4.1 Directory Path	43
5.1 Global Package Variables	44
5.2 Master_agent_config	46
5.3 Slave_agent_config	47
5.4 Env_config	47
6.1 Standalone Master Verification Plan	52
6.2 Standalone Slave Verification Plan	55
6.3 Back-to-Back Verification Plan	59
7.1 Assertion Table	6
9.1 Constraints in master and slave transactions	74
9.6 Test-package families	83
9.7 Regression testlists	83

List of Figures

Fig — Name of the figure	Pg. no
1.1 AXI Read and Write Channels	8
1.2 Read Channels of AXI	9
1.3 Write Channels of AXI	10
1.4 Burst types	15
1.5 Valid before Ready	18
1.6 Ready before Valid	18
1.7 Ready with Valid	19
1.8 Read transaction dependencies	19
1.9 Write transaction dependencies	20
1.10 Write response dependencies	21
2.1 AXI4 AVIP Testbench Architecture	22
3.1 AXI4 AVIP testbench block diagram	26
3.4 axi_write_address_channel_task	27
3.5 axi_write_data_channel_task	28
3.6 axi_write_response_channel_task	29
3.7 axi_read_address_channel_task	30
3.8 axi_read_data_channel_task	31
3.11 Slave driver proxy and BFM flow	32
3.12 write_address_phase	33
3.13 write_data_phase	34
3.14 write_response_phase	35
3.15 read_address_phase	36
3.16 read_data_phase (updated)	37
3.29 Master driver proxy run-phase	39
3.37 Slave driver proxy write flow	40
3.39 Slave driver proxy read flow	41
4.1 Package Structure of AXI4_AVIP	42
5.5.1 Memory mapping example	48
5.5.2 Global parameter declaration	48
5.5.3 Associative array declaration	49
5.5.4 Functions for memory mapping	49
5.5.5 Local variable declaration in function	49
5.5.6 Memory mapping in master agent configuration	50
5.5.7 Slave max/min address range	50
5.5.8 Memory mapping in slave agent configuration	51
7.1 Assertion code for stable signals check	61
7.2 Assertion code for unknown signals check	61
7.3 Assertion code for valid stable check	62
9.1 Test flow	70
9.2 AXI4 test cases flow chart	70
9.3 Constraint for write address	72
9.4 Constraint for write data	72
9.5 Constraint for read address	73

9.6 Constraint for memory	73
9.7 Constraints for read data response and wait states	74
9.8 do_copy method	76
9.9 do_compare method	77
9.10 do_print method	78
9.11 slave_tx do_copy method	79
9.12 slave_tx do_compare method	80
9.13 slave_tx do_print method	81

Table 7.1 Assertion Table

Assertion Label	Description
AXI_WA_STABLE_SIGNALS_CHECK	All signals must remain stable after AWVALID is asserted until AWREADY IS LOW
AXI_WA_UNKNOWN_SIGNALS_CHECK	A value of X on signals is not permitted when AWVALID is HIGH
AXI_WA_VALID_STABLE_CHECK	When AWVALID is asserted, then it must remain asserted until AWREADY is HIGH

Chapter 1

INTRODUCTION

AMBA (Advanced Microcontroller Bus Architecture) is open standard for communication and management of the functional blocks in SoC, provide different on chip communication protocols like CHI (Coherence Hub Interface), AXI (Advanced eXtensible Interface), ACE (Advanced Coherency Extension), AHB (Advanced High Performance Bus), APB (Advanced Peripheral Bus) developed by ARM (Advanced RISC Machine) . Flexibility of AMBA protocols is IP reuse for different SoC designs with different area, power and performance requirements.

As the AMBA protocols are widely used open standards which ensures compatibility between IPs of different suppliers for the SoC , with compatibility it enables low friction integration and reuse of IP which catalyse the faster time to market. The AMBA AXI protocol specification is defined to implement a high frequency, high bandwidth interface across a wide variety of applications in embedded, automotive and cellphones. It does not require complex bridge implementation for different peripheral devices. The AXI protocol includes some new features which extend previous versions and is compatible to complement CHI.

There are 3 types of AXI4-Interfaces (AMBA 4.0):

- AXI4 (Full AXI4): For high-performance memory-mapped requirements.
- AXI4-Lite: For simple, low-throughput memory-mapped communication (for example, to and from control and status registers).
- AXI4-Stream: For high-speed streaming data

1.1 AXI Read and Write Channels

The AXI protocol defines 5 channels:

- 2 are used for Read transactions
 - read address
 - read data
- 3 are used for Write transactions
 - Write address
 - Write data
 - Write response

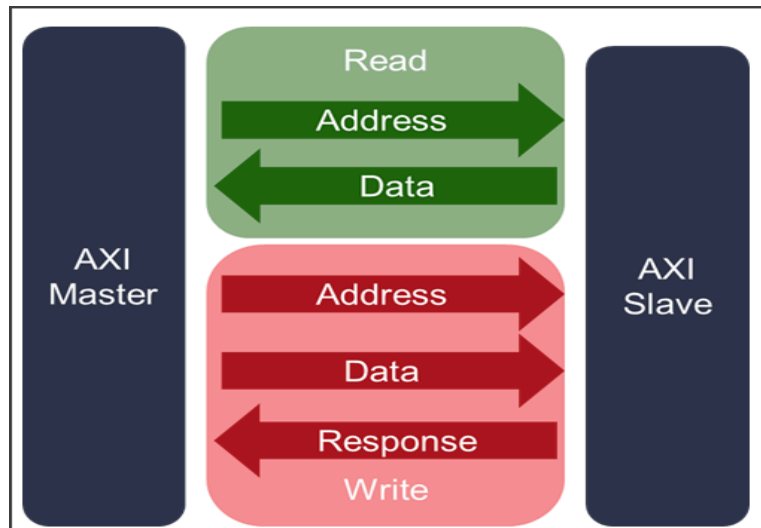


Fig 1.1 AXI Read and write Channels

The AXI is a burst-based protocol that defines the following transaction channels independently:

1. Address Read(AR)
2. Read data(R)
3. Address Write(AW)
4. Write data(W)
5. Write response(B)

Control information about the kind of data to be delivered is carried by an address channel. AXI protocol entails the following steps:

1. Enables the distribution of address information before the actual data transmission
2. Supports a large number of open transactions
3. Allows transactions to be completed out of order

A channel is an independent collection of AXI signals associated with the VALID and READY signals.

Note: An AXI4/AXI3/AXI4-Lite Interface can be read only (only includes the 2 Read channels) or write only (only includes the 3 Write channels).

A piece of data transmitted on a single channel is called a transfer. A transfer happens when both the VALID and READY signals are high while there is a rising edge of the clock.

1.2 AXI Read Transactions

An AXI Read transaction requires multiple transfers on the 2 Read channels.

- First, the **Address Read Channel** is sent from the Master to the Slave to set the address and some control signals.
- Then the data for this address is transmitted from the Slave to the Master on the **_Read data channel**.

Note that, as per the figure below, there can be multiple data transfers per address. This type of transaction is called a **burst**.

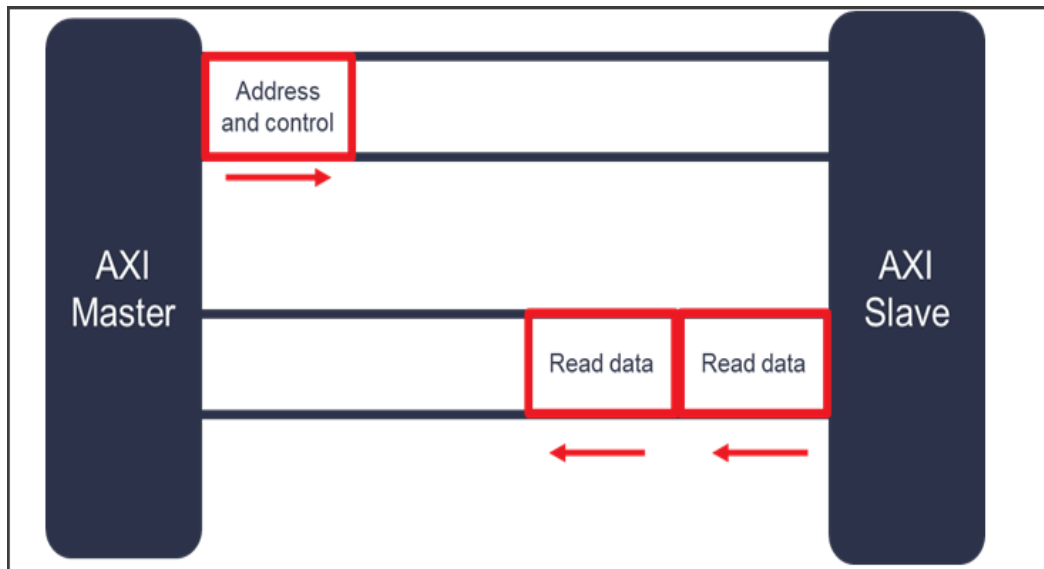


Fig 1.2 Read Channels of AXI

1.3 AXI Write Transactions

An AXI Write transaction requires multiple transfers on the 3 Read channels.

- First, the **Address Write Channel** is sent Master to the Slave to set the address and some control signals.
- Then the data for this address is transmitted Master to the Slave on the **Write data channel**.
- Finally the write response is sent from the Slave to the Master on the **Write Response Channel** to indicate if the transfer was successful.

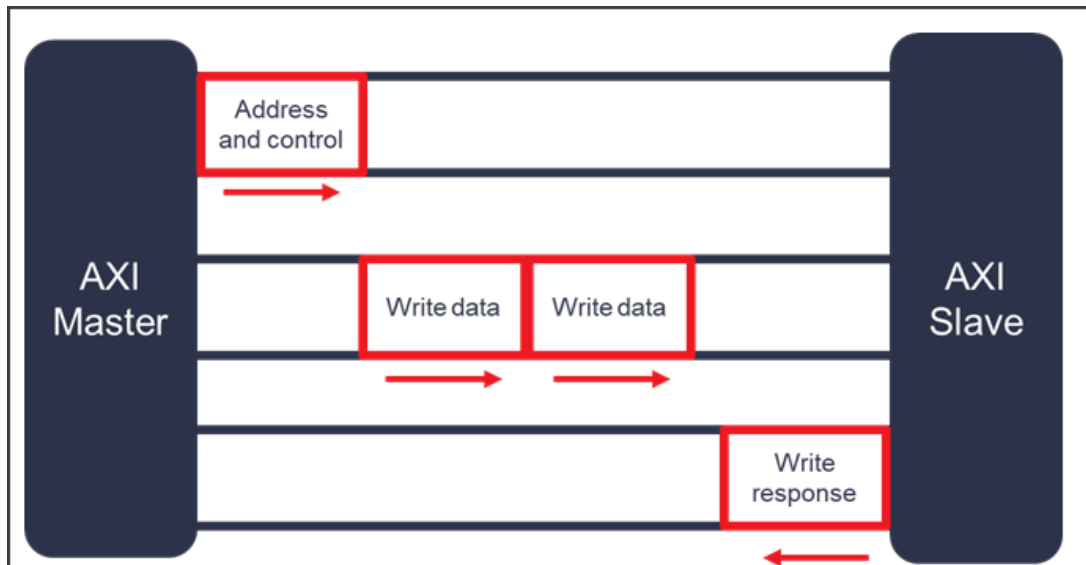


Fig 1.3 Write Channels of AXI

The possible response values on the Write Response Channel are:

- OKAY (0b00): Normal access success. Indicates that a normal access has been successful
- EXOKAY (0b01): Exclusive access okay.
- SLVERR (0b10): Slave error. The slave was reached successfully but the slave wishes to return an error condition to the originating master (for example, data read not valid).
- DECERR (0b11): Decode error. Generated, typically by an interconnect component, to indicate that there is no slave at the transaction address

Note: Read transactions also have a response value but this response is transmitted as part of the Read Response Channel

+

1.4 Interface Signal Definition

Table 1.1: Write Address Signals

Signal	Source	Supporting Version	Definition of Signal
AWID[x:0]	Manager / Master	AXI3 and AXI4	Provides ID for each transaction
AWADDR[31:0]	Manager / Master	AXI3 and AXI4	Address for write request
AWLEN[7:0]	Manager / Master	AXI4	Number of transfers support in each transaction
AWSIZE[2:0]	Manager / Master	AXI3 and AXI4	No. of bytes to be transfer in each beat
AWBURST[1:0]	Manager / Master	AXI3 and AXI4	Indicates type of burst to be performed
AWLOCK	Manager / Master	AXI4	Indicates the atomic characteristics of the transaction
AWCACHE[3:0]	Manager / Master	AXI3 and AXI4	This signal indicates the system performance
AWPROT[2:0]	Manager / Master	AXI3 and AXI4	Provides system level security and privileged access to each transaction.
AWQoS[3:0]	Manager / Master	AXI4	Use to prioritise the transactions
AWREGION[3:0]	Manager / Master	AXI4	Region identifier
AWVALID	Manager / Master	AXI3 and AXI4	Use to validate the associated signal inorder to pass valid information.

AWREADY	Subordinate/slave	AXI3 and AXI4	Indicates weather slave is ready for the transactions.
---------	-------------------	---------------	--

Table 1.2: Write data Signals

Signal	Source	Supporting Version	Definition of Signal
AWDATA[x:0]	Manager / Master	AXI3 and AXI4	Write data signal
AWSTRB[x:0]	Manager / Master	AXI3 and AXI4	Used to send valid bytes for each transfer
AWLAST	Manager / Master	AXI3 and AXI4	Indicates last transfer for the transaction
AWVALID	Manager / Master	AXI3 and AXI4	Used to validate the associated signal in order to pass valid information.
AWREADY	Subordinate/slave	AXI3 and AXI4	Indicates weather slave is ready for the transactions.

Table 1.3 : Write Response Signals

Signal	Source	Supporting Version	Definition of Signal
BID[x:0]	Subordinate/slave	AXI3 and AXI4	Provides ID for each write transaction
BRSEP[1:0]	Subordinate/slave	AXI3 and AXI4	Write Response signal
BVALID	Subordinate/slave	AXI3 and AXI4	Use to validate the associated signal in order to pass valid information.
BREADY	Manager / Master	AXI3 and AXI4	Indicates weather master is ready for the transactions.

Table 1.4 : Read Address Signals

Signal	Source	Supporting Version	Definition of Signal
ARID[x:0]	Manager / Master	AXI3 and AXI4	Provides ID for each transaction
ARADDR[31:0]	Manager / Master	AXI3 and AXI4	Address for write request
ARLEN[7:0]	Manager / Master	AXI4	No.of transfers support in each transaction
ARSIZE[2:0]	Manager / Master	AXI3 and AXI4	No. of bytes to be transfer in each beat
ARBURST[1:0]	Manager / Master	AXI3 and AXI4	Indicates type of burst to be performed
ARLOCK	Manager / Master	AXI4	Indicates the atomic characteristics of the transaction
ARCACHE[3:0]	Manager / Master	AXI3 and AXI4	This signal indicates the system performance
ARPROT[2:0]	Manager / Master	AXI3 and AXI4	Provides system level security and privileged access to each transaction.
ARQoS[3:0]	Manager / Master	AXI4	Use to prioritise the transactions
ARREGION[3:0]	Manager / Master	AXI4	Region identifier
ARVALID	Manager / Master	AXI3 and AXI4	Use to validate the associated signal in order to pass valid information.
ARREADY	Subordinate/slave	AXI3 and AXI4	Indicates whether slave is ready for the transactions.

Table 1.5 : Read data Signals

Signal	Source	Supporting Version	Definition of Signal
RID[x:0]	Subordinate/ slave	AXI3 and AXI4	Provides ID for each read transaction
RDATA[x:0]	Subordinate/ slave	AXI3 and AXI4	Read data signal
RESP[1:0]	Subordinate/ slave	AXI3 and AXI4	Read response signal
RLAST	Subordinate/ slave	AXI3 and AXI4	Indicates last transfer for the transaction
RVALID	Subordinate/ slave	AXI3 and AXI4	Use to validate the associated signal in order to pass valid information.
READY	Manager / Master	AXI3 and AXI4	Indicates whether master is ready for the transactions.

1.5 Key Features

1. Axi4 supports write and read in parallel
2. Blocking and Non Blocking Transfers
3. Separate address/control and data phases
4. Support for unaligned data transfers, using byte strobes
5. Uses burst-based transactions with only the start address issued
6. Support for issuing multiple outstanding addresses
7. Support for narrow transfers

TO_DO Key features :

- 1.Out-of-Order
2. Low-power features

1.6 Reset

Reset assertion is asynchronous to clock and deassertion is synchronous to clock

1.7 Write Channel Signal Descriptions :

Awid : Master - Write Address ID

1. Each transaction has its own id.

Awaddr : Master - Write Address

1. Write address for the first **transfer**.
2. Drives Address of the first-byte in the tx to the slave.
3. The slave must calculate the subsequent transfers in the burst.
4. **Burst must not cross a 4KB address boundary.**

Awlength : Master - Burst Length

1. Gives the exact number of transfers in a burst.
2. Burst_length = AWLEN + 1
3. AWLEN[7:0], for write transfers for INCR burst(new feature for AXI4)
4. AWLEN[3:0], for write transfers for FIXED, WRAP burst
5. AWLEN for different burst types :
 - a. FIXED : 1 to 16
 - b. INCR : 1 to 256 transfers
 - c. WRAP : 2,4,8 or 16
6. Early termination of burst is not supported.
 - a. Hence, master can reduce data transfer in write burst by deasserting pstobe.

Starting address: 0x1004			
Transfer size: 4 Bytes			
Transfer length: 4 beats			
1 st beat	0x1004	0x1004	0x1004
2 nd beat	0x1004	0x1008	0x1008
3 rd beat	0x1004	0x100C	0x100C
4 th beat	0x1004	0x1010	0x1000
	FIXED	INCR	WRAP

Fig 1.4 Burst types

Awsiz : Master - Size of each transfer

1. Gives the size of each transfer in the burst.

AxSIZE[2:0] Bytes in transfer

0b000	1
0b001	2
0b010	4
0b011	8
0b100	16
0b101	32
0b110	64
0b111	128

2. No of address bytes of transfer in the burst will be calculated as two powers of AWSIZE of Address Channel bus.
3. If the AXI bus is wider than the burst size, the AXI interface must determine from the transfer address which byte lanes of the data bus to use for each transfer. See Data read and write structure.
4. The size of any transfer must not exceed the data bus width of either agent in the transaction.

Awburst: MASTER - Type of transfer

fixed :

1. The start address is the same for all transfers in the burst.
2. The byte lanes still may differ based on the assertion ofWSTRB
3. This burst type is used for repeated accesses to the same location such as when loading or emptying a FIFO.

Incr :

1. The address will be incremented for the next beat/transfer from the start address.
2. The increment will be based on the size of the transfer.
3. This burst type is used for access to normal sequential memory.

wrap :

1. A wrapping burst is similar to an incrementing burst, except that the address wraps around to a lower address if an upper address limit is reached.
2. Restrictions to wrap burst :
 - a. Start address must be aligned to the size of each transfer.
 - b. Length of bursts must be 2,4,8 or 16b transfers
3. Lowest address used by burst is aligned to the total size of the data to be transferred.
 - a. Lowest address is the **wrap boundary**.
 - b. **Wrap Boundary** = size of each transfer on the burst * total num of transfers
4. It increments till the wrap boundary from the start address, i.e.,
 - a. **Incremented address** = wrap_boundary + total size of data to be transferred
5. The start address can be higher than the wrap boundary. This means that the address wraps for any WRAP burst for which the first address is higher than the wrap boundary.
6.

AxBURST[1:0]	Burst type
0b00	FIXED
0b01	INCR
0b10	WRAP
0b11	Reserved

Write Data channel

1. Data bus width : 8,16,32,64,128,256,512,1024 bits wide
2. Different bus width with for write and read data channels
3. Write strobe feature
4. Different combination of strobes
5. Valid scenario - 8bits data on 32 bits data bus
6. Invalid scenarios - 16bits data on 32 bits data bus but strobes like 111 - invalid

Note: The read channel doesn't require buffer because the data sent from slave with the ID will be routed correctly by the interconnect. Hence, no buffer on this side.

Write Response

1. Valid default value is 0, other signals can be anything
2. Ready can be low or high in default state.
3. It should be high, only if master can accept the response in 1 clock cycle - transfer in 1 clock cycle
4. If low, it takes 2 clock cycles min.
5. Response is driven only after the write-data transaction is completed.

Read Address:

1. Valid default value is 0, other signals can be anything
2. Ready can be low or high in default state.
3. If high, it should be able to accept the data.- transfer in 1 clock cycle
4. If low, it takes 2 clock cycles min. Hence, not recommended but based on implementation

Read Response:

1. Valid default value is 0, other signals can be anything
2. Ready can be low or high in default state.
3. If high, master should be able to accept the data.- transfer in 1 clock cycle
4. If low, it takes 2 clock cycles min.
5. Response, along with data, is driven only after the read address transaction is completed.

1.8 Handshake Process:

All five transaction channels use the same VALID/READY handshake process to transfer address, data, and control information. This two-way flow control mechanism means both the master and slave can control the rate at which the information moves between master and slave. The source generates the VALID signal to indicate when the address, data or control

information is available. The destination generates the READY signal to indicate that it can accept the information. Transfer occurs only when both the VALID and READY signals are HIGH.

In Figure below the source presents the address, data or control information after T1 and asserts the VALID signal. The destination asserts the READY signal after T2, and the source must keep its information stable until the transfer occurs at T3, when this assertion is recognized.

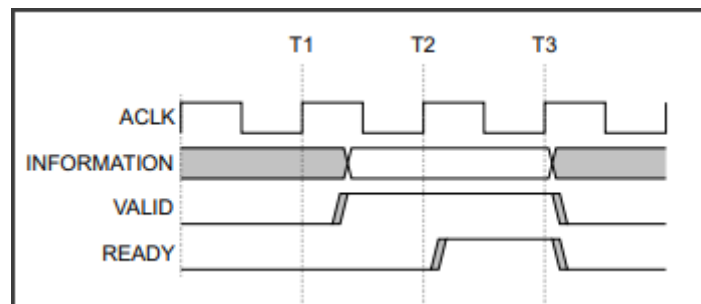


Fig 1.5 Valid before Ready

Once VALID is asserted it must remain asserted until the handshake occurs, at a rising clock edge at which VALID and READY are both asserted.

In Figure below , the destination asserts READY, after T1, before the address, data or control information is valid, indicating that it can accept the information. The source presents the information, and asserts VALID, after T2, and the transfer occurs at T3, when this assertion is recognized. In this case, transfer occurs in a single cycle

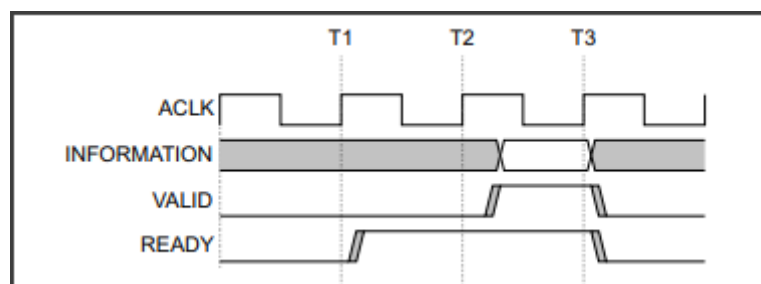


Fig 1.6 Ready before valid

A destination is permitted to wait for VALID to be asserted before asserting the sserted, it is permitted to deassert REcorresponding READY. If READY is asADY before VALID is asserted.

In Figure below both the source and destination happen to indicate, after T1, that they can transfer the address, data or control information. In this case the transfer occurs at the rising

clock edge when the assertion of both VALID and READY can be recognized. This means the transfer occurs at T2.

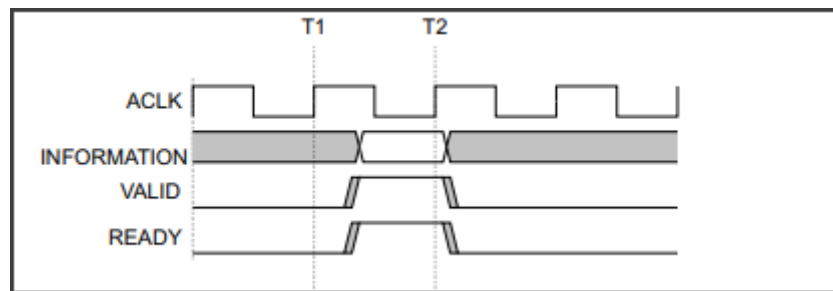


Fig 1.7 Ready with valid

1.9 Relationships between the channels:

Read transaction dependencies

Figure below shows the read transaction handshake signal dependencies, and shows that, in a read transaction:

1. The master must not wait for the slave to assert ARREADY before asserting ARVALID
2. The slave can wait for ARVALID to be asserted before it asserts ARREADY
3. The slave can assert ARREADY before ARVALID is asserted
4. The slave must wait for both ARVALID and ARREADY to be asserted before it asserts RVALID to indicate that valid data is available
5. The slave must not wait for the master to assert RREADY before asserting
6. The master can wait for RVALID to be asserted before it asserts RREADY
7. The master can assert RREADY before RVALID is asserted.

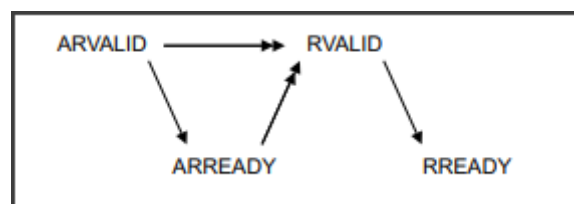


Fig 1.8 Read transaction dependencies

Write transaction dependencies

1. The master must not wait for the slave to assert AWREADY or WREADY before asserting AWVALID or WVALID
2. The slave can wait for AWVALID or WVALID, or both before asserting AWREADY

3. The slave can assert **AWREADY** before **AWVALID** or **WVALID**, or both, are asserted
4. The slave can wait for **AWVALID** or **WVALID**, or both, before asserting **WREADY**
5. The slave can assert **WREADY** before **AWVALID** or **WVALID**, or both, are asserted
6. The slave must wait for both **WVALID** and **WREADY** to be asserted before asserting **BVALID** the slave must also wait for **WLAST** to be asserted before asserting **BVALID**, because the write response, **BRESP**, must be signaled only after the last data transfer of a write transaction
7. The slave must not wait for the master to assert **BREADY** before asserting **BVALID**
8. The master can wait for **BVALID** before asserting **BREADY**
9. The master can assert **BREADY** before **BVALID** is asserted.

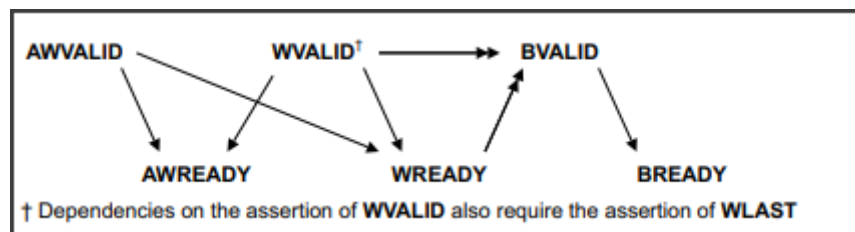
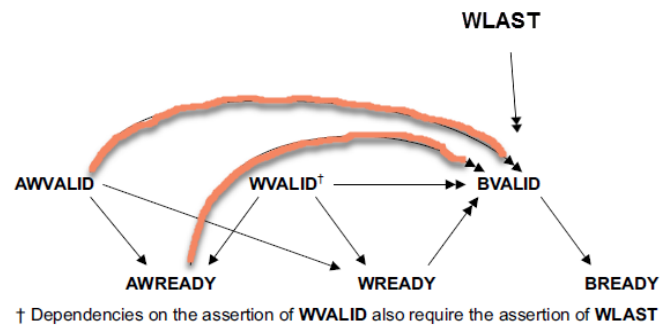


Fig 1.9 Write transaction dependencies

Write Response dependencies

The master must not wait for the slave to assert **AWREADY** or **WREADY** before asserting **AWVALID** or **WVALID**

1. The slave can wait for **AWVALID** or **WVALID**, or both, before asserting **AWREADY**
2. The slave can assert **AWREADY** before **AWVALID** or **WVALID**, or both, are asserted
3. The slave can wait for **AWVALID** or **WVALID**, or both, before asserting **WREADY**
4. The slave can assert **WREADY** before **AWVALID** or **WVALID**, or both, are asserted
5. The slave must wait for **AWVALID**, **AWREADY**, **WVALID**, and **WREADY** to be asserted before asserting **BVALID** the slave must also wait for **WLAST** to be asserted before asserting **BVALID** because the write response, **BRESP** must be signaled only after the last data transfer of a write transaction
6. The slave must not wait for the master to assert **BREADY** before asserting **BVALID**
7. The master can wait for **BVALID** before asserting **BREADY**
8. The master can assert **BREADY** before **BVALID** is asserted.



\ Fig 1.10 Write response dependencies

Chapter 2

ARCHITECTURE

2.1 AXI4 AVIP Testbench Architecture

The accelerated VIP is divided into the two top modules as HVL and HDL top as shown in fig 2.1. The whole idea of using Accelerated VIP is to push the synthesizable part of the testbench into the separate top module along with the interface and it is named as HDL TOP. and the unsynthesizable part is pushed into the HVL TOP it provides the ability to run the longer tests quickly. This particular testbench can be used for the simulation as well as the emulation based on the mode of operation.

HVL TOP has the design which is untimed and the transactions flow from both master virtual sequence and slave virtual sequence onto the AXI4 I/F through the BFM Proxy and BFM and gets the data from monitor BFM and uses the data to do checks using scoreboard and coverage

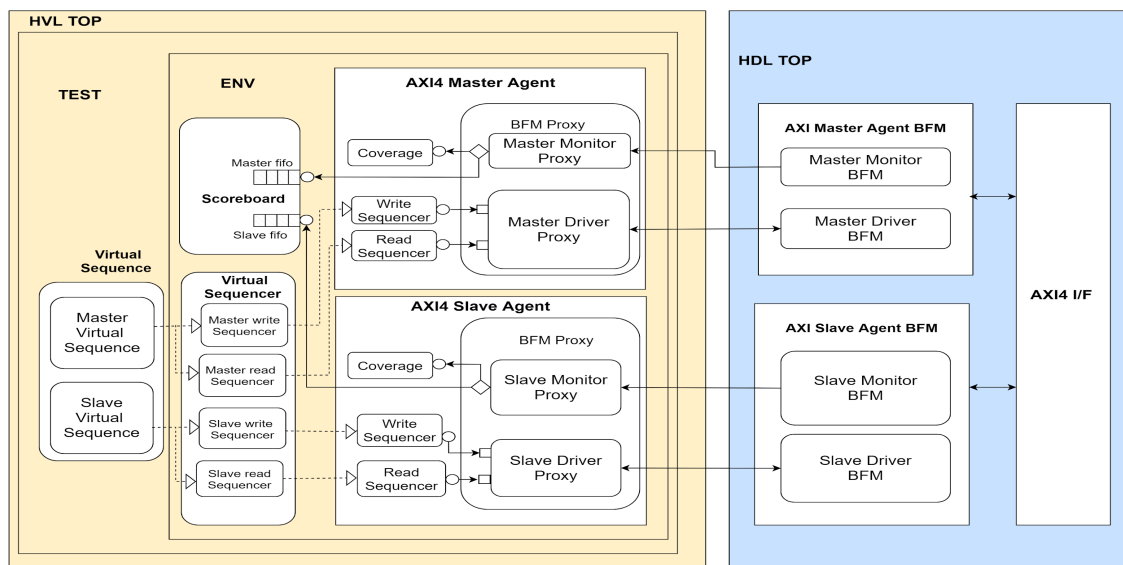


Fig 2.1 AXI4 AVIP Testbench Architecture

HDL TOP consists of the design part which is timed and synthesizable, Clock and reset signals are generated in the HDL TOP. Bus Functional Models (BFMs) i.e synthesizable part of drivers and monitors are present in HDL TOP, proxy. BFMs also have the back pointers to it's proxy to call non-blocking methods which are defined in the

We have the tasks and functions within the drivers and monitors which are called by the driver and monitor proxy inside the HVL. This is how the data is transferred between the HVL TOP and HDL TOP.

HDL and HVL use transaction based communication to enable the information rich transactions and since clock is generated within the HDL TOP inside the emulator it allows the emulator to run at full speed.

..

Chapter 3

IMPLEMENTATION

3.1 Pin Interface

Table 3.1 shows the AXI4 pins used to interface to external device

Signal	Master Direction	Slave Direction	Width of signals	Description
aclk	input	input	1 bit	System generated clock
areset_n	input	input	1 bit	It is an active low reset generated by system
awid	output	input	4 bits	This signal is the identification tag for the write address group of signals.
awaddr	output	input	32 bits	The write address gives the address of the first transfer in a write burst transaction.
awlen	output	input	4 bits	The burst length gives the exact number of transfers in a burst. This information determines the number of data transfers associated with the address.
awsize	output	input	3 bits	This signal indicates the size of each transfer in the burst.
awburst	output	input	2 bits	The burst type and the size information, determine how the address for each transfer within the burst is calculated.
awlock	output	input	2 bits	Provides additional information about the atomic characteristics of the transfer.
awcache	output	input	4 bits	This signal indicates how transactions are required to progress through a system.
awprot	output	input	3 bits	This signal indicates the privilege and security level of the transaction, and whether the transaction is a data access or an instruction access.
awqos	output	input	4 bits	The qos identifier is sent for each write transaction.
awregion	output	input	4 bits	Permits a single physical interface on a slave to be used for multiple logical interfaces.
awuser	output	input	4 bits	Optional User-defined signal in the write address channel.
awvalid	output	input	1 bit	This signal indicates that the channel is signalling valid write address and control information.
awready	input	output	1 bit	This signal indicates that the slave is ready to accept an address and associated control signals.
wdata	output	input	32 bits	Write data.
wstrb	output	input	4 bits	This signal indicates which byte lanes hold valid data. There is one write strobe bit for each eight bits

				of the write data bus.
wlast	output	input	1 bit	This signal indicates the last transfer in a write burst.
wuser	output	input	4 bits	Optional User-defined signal in the write data channel.
wvalid	output	input	1 bit	This signal indicates that valid write data and strobes are available.
wready	input	output	1 bit	This signal indicates that the slave can accept the write data.
bid	input	output	4 bits	This signal is the ID tag of the write response.
bresp	input	output	2 bits	This signal indicates the status of the write transaction.
buser	input	output	4 bits	Optional User-defined signal in the write response channel.
bvalid	input	output	1 bit	This signal indicates that the channel is signalling a valid write response.
bready	output	input	1 bit	This signal indicates that the master can accept a write response.
arid	output	input	4 bits	This signal is the identification tag for the read address group of signals.
araddr	output	input	32 bits	The read address gives the address of the first transfer in a read burst transaction.
arlen	output	input	8 bits	This signal indicates the exact number of transfers in a burst.
arsize	output	input	3 bits	This signal indicates the size of each transfer in the burst.
arburst	output	input	2 bits	The burst type and the size information determine how the address for each transfer within the burst is calculated.
arlock	output	input	2 bits	This signal provides additional information about the atomic characteristics of the transfer.
arcache	output	input	4 bits	This signal indicates how transactions are required to progress through a system.
arprot	output	input	3 bits	This signal indicates the privilege and security level of the transaction, and whether the transaction is a data access or an instruction access.
arqos	output	input	4 bits	qos identifier sent for each read transaction.
arregion	output	input	4 bits	Permits a single physical interface on a slave to be used for multiple logical interfaces.
aruser	output	input	4 bits	Optional User-defined signal in the read address channel.
arvalid	output	input	1 bit	This signal indicates that the channel is signalling valid read address and control information.

3.2 Testbench Components

This section describes each AXI4 AVIP testbench component following the original document structure. The component hierarchy is shown in Fig. 3.1. Where a component exposes TLM connectivity (analysis ports, analysis FIFOs and TLM FIFOs) the current declarations and connections are shown. Two enumerations and a parameter in `axi4_globals_pkg` drive the transfer and response behaviour:

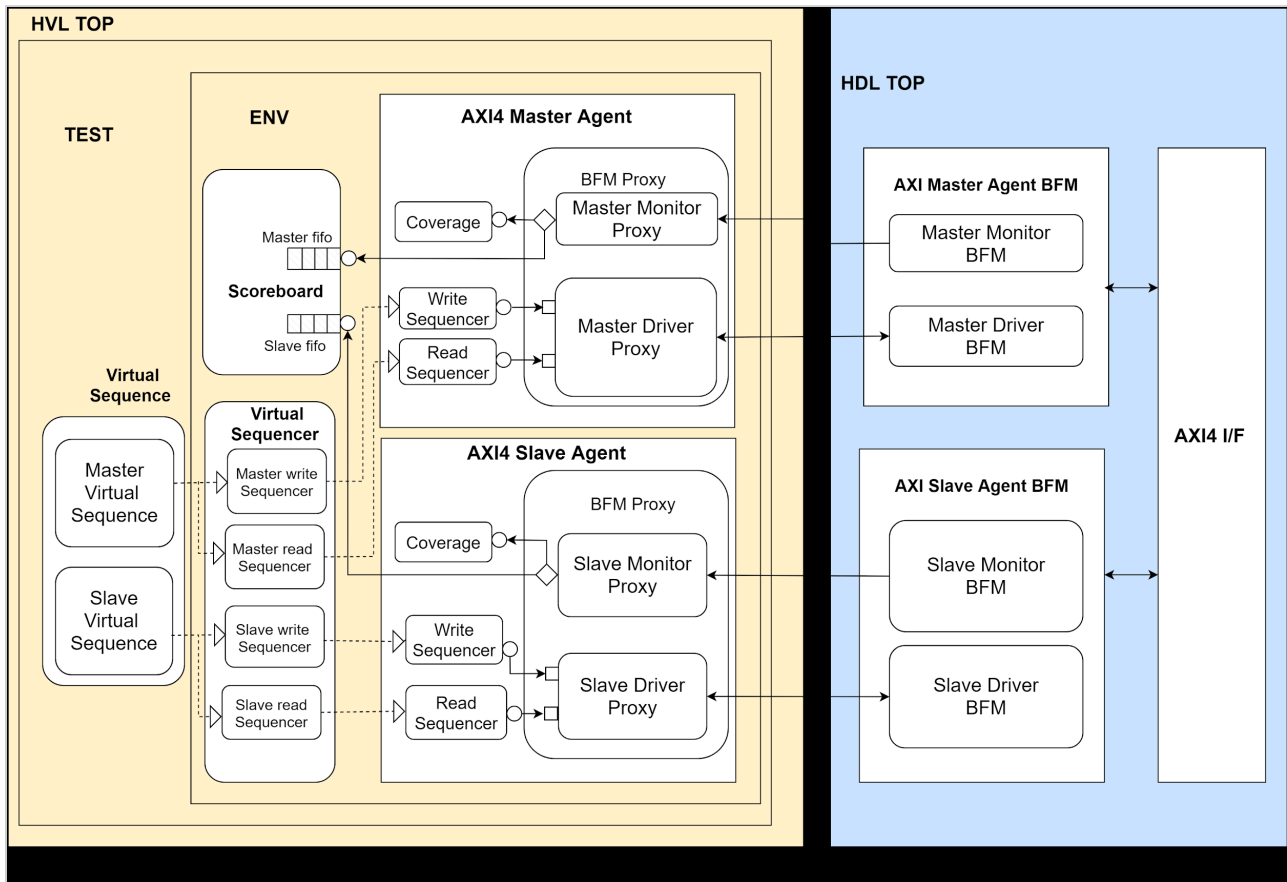


Fig. 3.1 AXI4 AVIP testbench block diagram (HVL_TOP and HDL_TOP)

Type / parameter	Values / default	Purpose
transfer_type_e	OUTSTANDING_WRITE, OUTSTANDING_READ, NON_OUTSTANDING_WRITE, NON_OUTSTANDING_READ	Whether the master overlaps multiple transfers or issues one at a time.
response_mode_e	RESP_IN_ORDER, RESP_OUT_OF_ORDER	How the slave orders its responses.
OUTSTANDING_FIFO_DEPTH	16	Sizing for outstanding-transaction buffering.

3.2.1 AXI Hdl Top

The synthesizable top module. It instantiates the AXI4 interface, the master and slave agent BFMs and the protocol assertion modules, generates `ac1k/aresetn`, and passes the virtual interface handles into

the UVM environment through the config DB.

3.2.2 AXI Interface

A single clocked `axi4_if` carries all five AXI channels (write-address, write-data, write-response, read-address, read-data). The BFM's drive/sample it and the assertions observe it.

3.2.3 AXI Master Agent BFM Module

Instantiates the master driver BFM and master monitor BFM and binds them to the interface, exposing the BFM handles to the UVM master agent through the config DB.

3.2.4 AXI Master Driver BFM Interface

The master driver BFM drives the five AXI channels through per-channel tasks. Each task drives its channel signals, asserts `*valid`, waits for the corresponding `*ready` handshake and then deasserts. The flowcharts below describe these channel tasks.

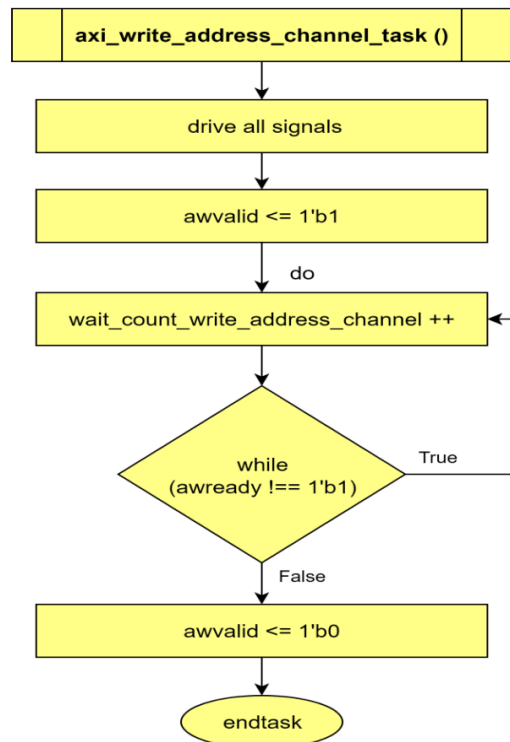


Fig. 3.4 `axi_write_address_channel_task`

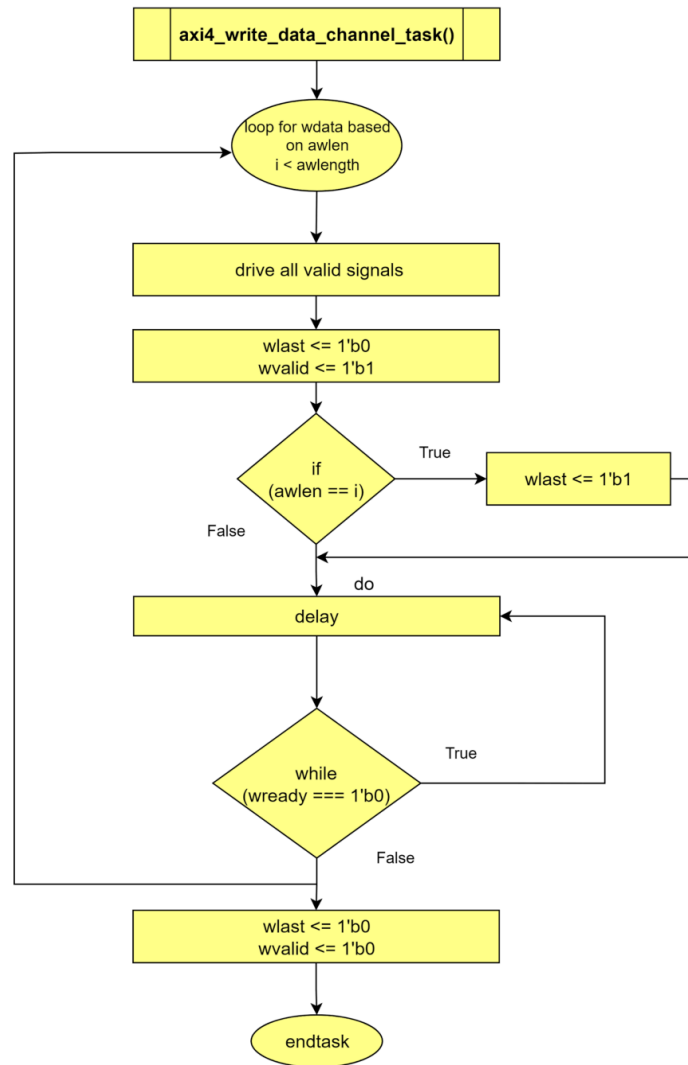


Fig. 3.5 axi_write_data_channel_task

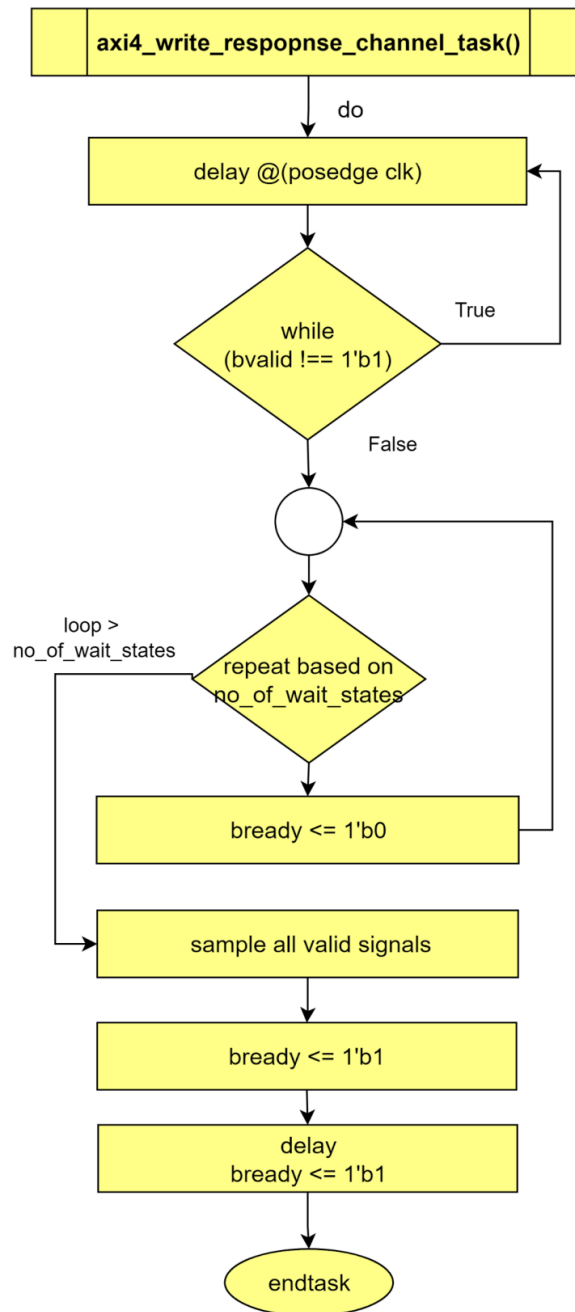


Fig. 3.6 axi_write_response_channel_task

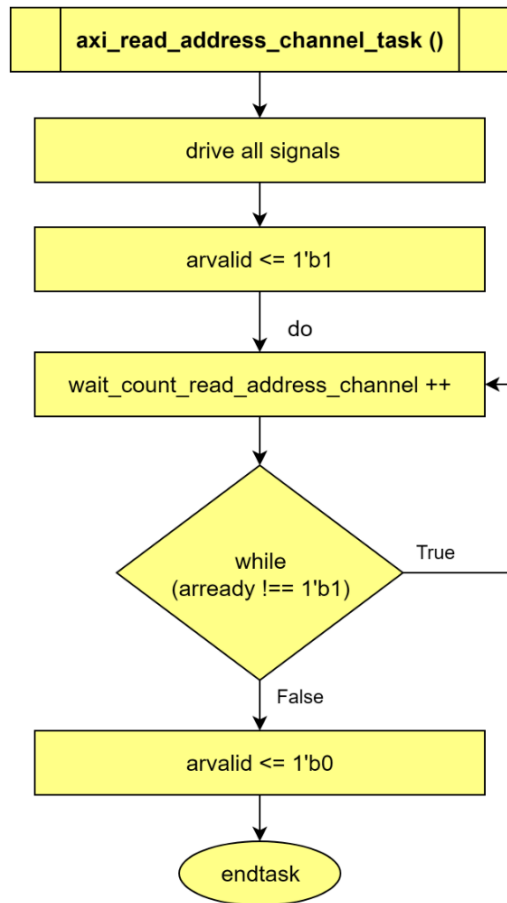


Fig. 3.7 *axi_read_address_channel_task*

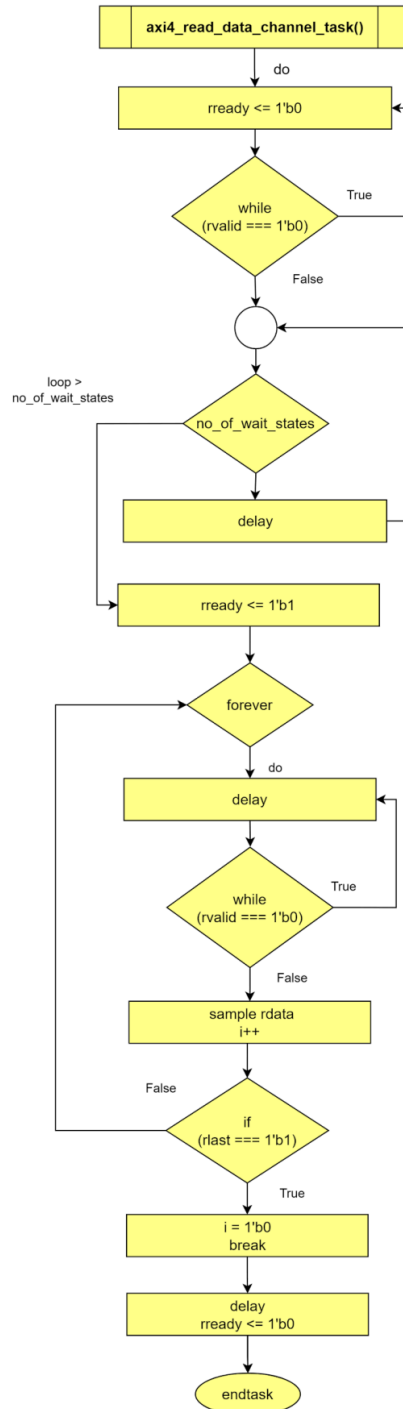


Fig. 3.8 axi_read_data_channel_task

3.2.5 AXI Master Monitor BFM Interface

The master monitor BFM samples completed transfers on each channel and forwards them to the master monitor proxy for publication on its analysis ports (see §3.2.16).

3.2.6 AXI Slave Agent BFM Module

Instantiates the slave driver BFM and slave monitor BFM and binds them to the interface, exposing the BFM handles to the UVM slave agent.

3.2.7 AXI Slave Driver BFM Interface

The slave driver BFM responds on the five AXI channels through per-phase tasks. The flowcharts below describe these phases. **The read-data phase flowchart has been updated** for the current BFM: the wait for `rready` is now X-safe (`while(rready===1'b0 || $isunknown(rready))`); the other phases are unchanged.

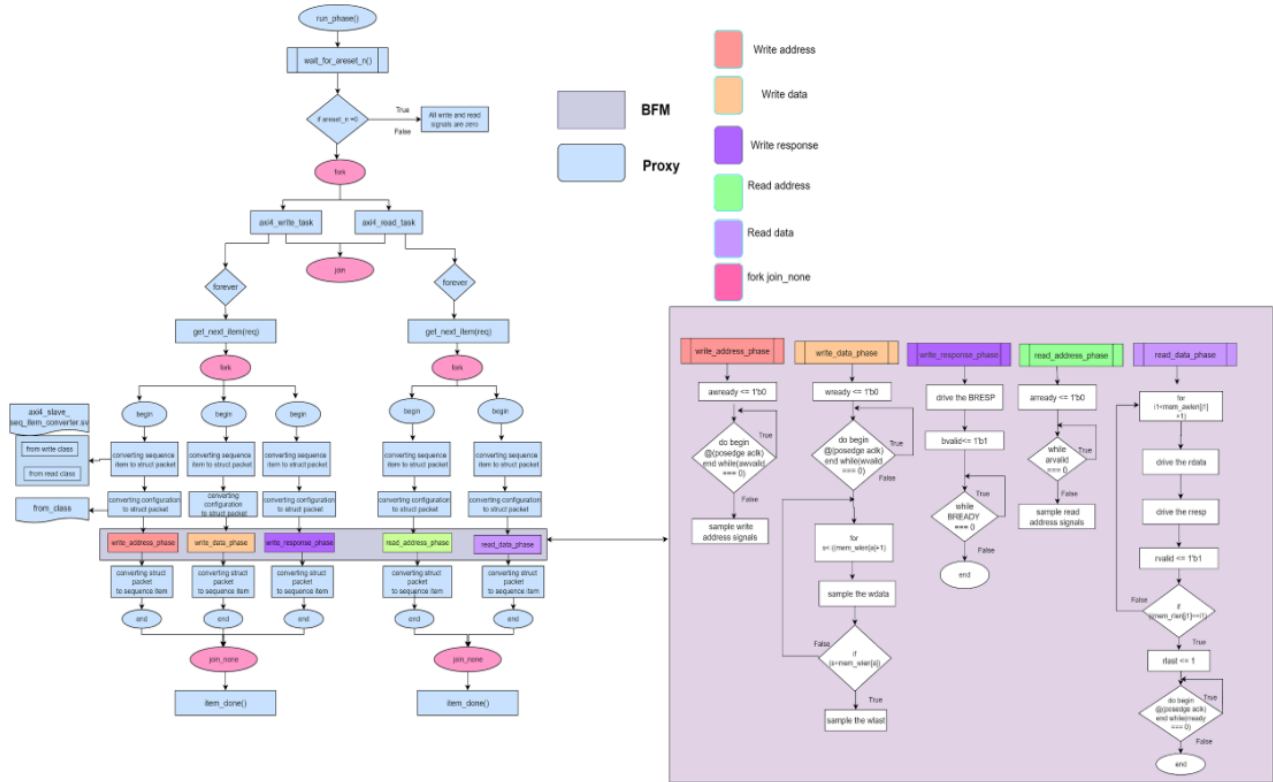


Fig. 3.11 Slave driver proxy and BFM flow

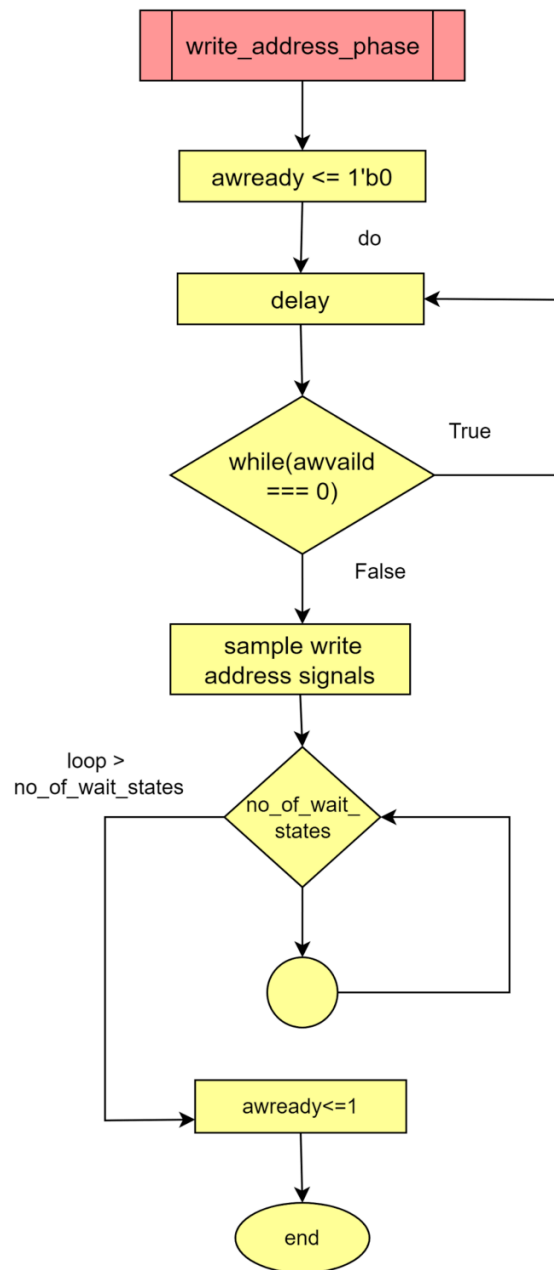


Fig. 3.12 write_address_phase

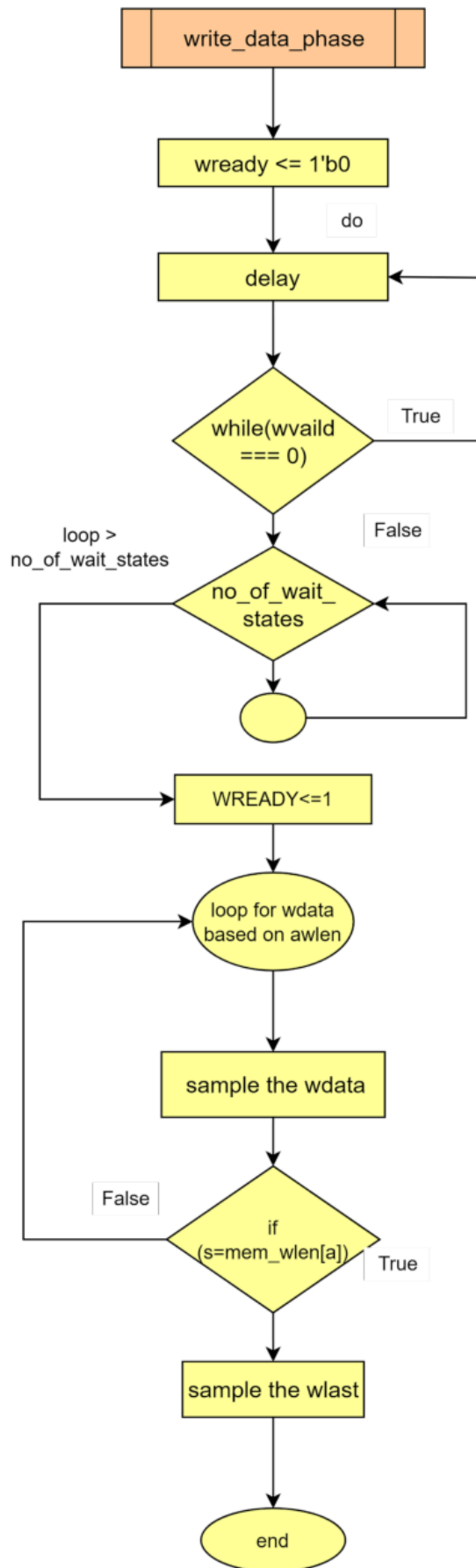


Fig. 3.13 write_data_phase

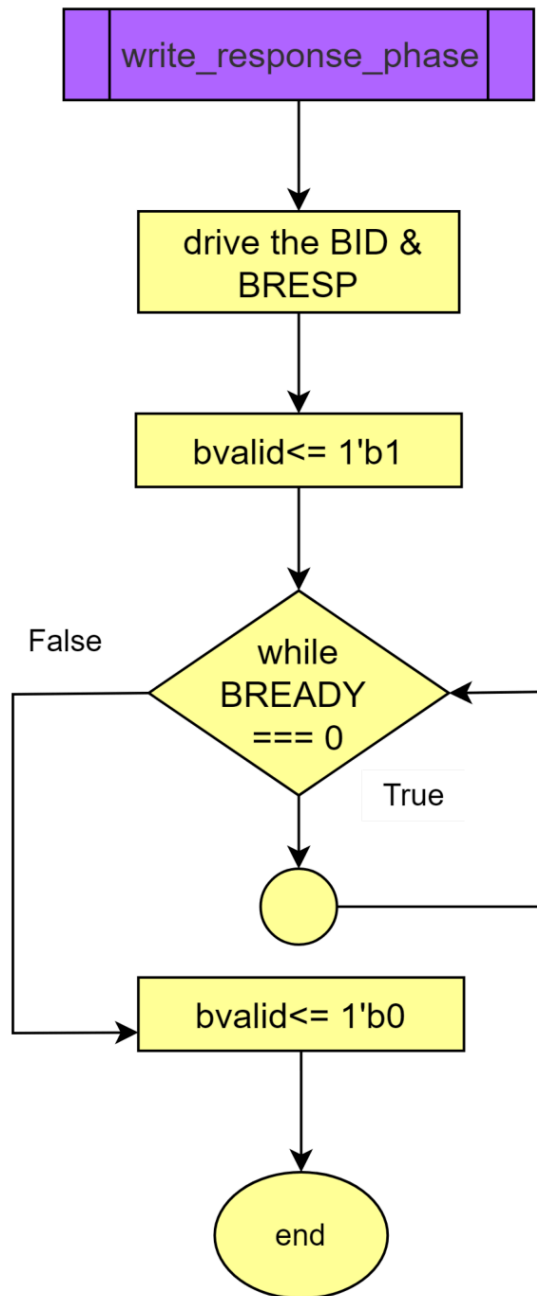


Fig. 3.14 write_response_phase

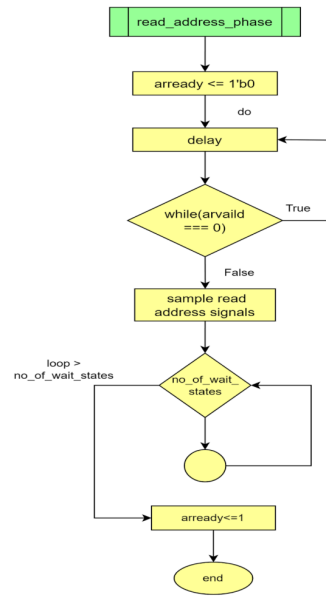


Fig 3.15 Read_address_phase

Fig. 3.15 read_address_phase

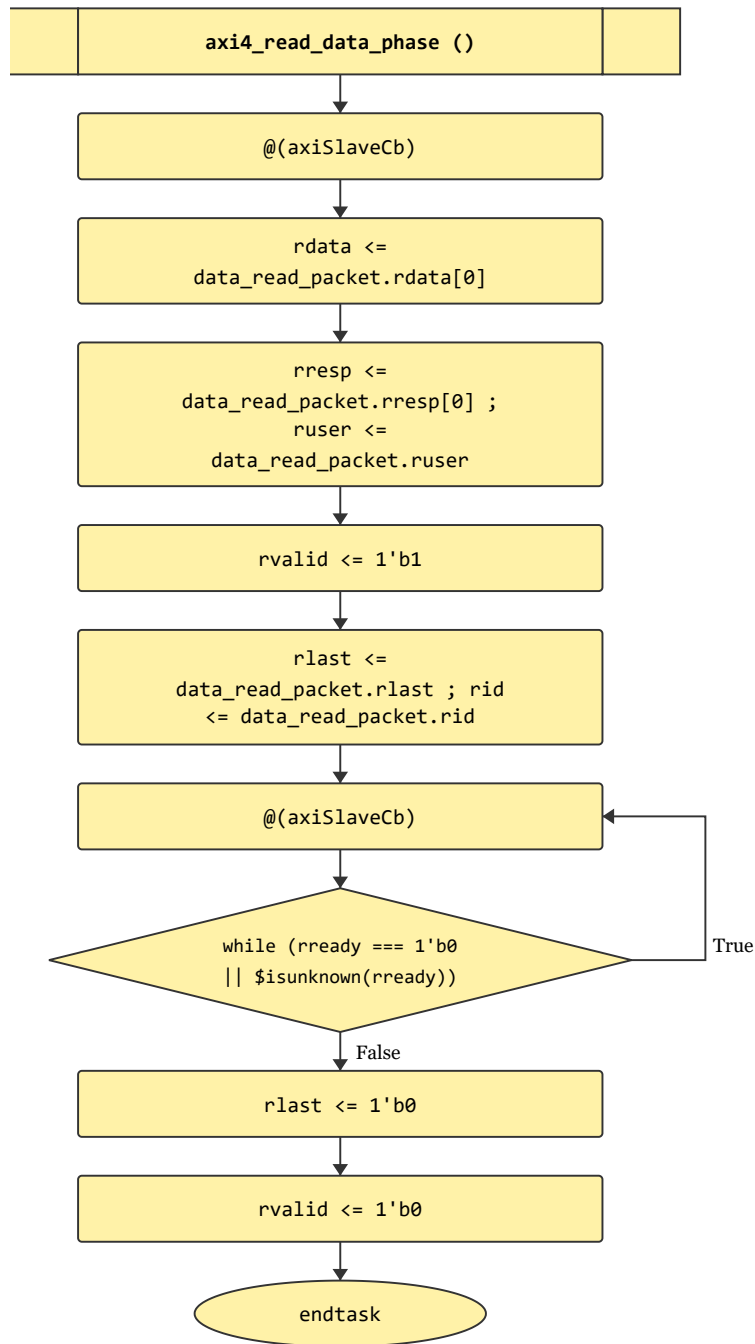


Fig. 3.16 read_data_phase (updated: X-safe rready wait)

3.2.8 AXI Slave Monitor BFM Interface

The slave monitor BFM samples completed transfers and forwards them to the slave monitor proxy (see §3.2.20).

3.2.9 AXI HVL_TOP

The UVM top that builds the environment and starts the selected test; it hosts the agents, the environment, the scoreboard and the virtual sequencer.

3.2.10 AXI Environment

The environment (`axi4_env`) constructs the agents and the scoreboard and, in its `connect_phase`, wires every monitor-proxy analysis port to the matching scoreboard endpoint. Address and write-data channels connect to analysis *imps* directly; read-data and write-response connect through the analysis FIFO's `analysis_export`:

```
function void axi4_env::connect_phase(uvm_phase phase);
  // master monitor ports -> scoreboard
  mon.axi4_master_write_address_analysis_port
    .connect(sb.axi4_master_write_address_analysis_fifo);
  mon.axi4_master_write_data_analysis_port
    .connect(sb.axi4_master_write_data_analysis_fifo);
  mon.axi4_master_write_response_analysis_port
    .connect(sb.axi4_master_write_response_analysis_fifo.analysis_export);
  mon.axi4_master_read_address_analysis_port
    .connect(sb.axi4_master_read_address_analysis_fifo);
  mon.axi4_master_read_data_analysis_port
    .connect(sb.axi4_master_read_data_analysis_fifo.analysis_export);
  // slave monitor ports -> scoreboard : same five channels, axi4_slave_tx
endfunction
```

3.2.11 AXI Scoreboard

The scoreboard receives every monitored transfer and compares master activity against slave activity. Each side exposes **five** endpoints: the address and write-data channels use custom analysis imps (``uvm_analysis_imp_decl`, each with its own `write_*`() callback), and the read-data and write-response channels use `uvm_tlm_analysis_fifos`. Matching is done by transaction ID so outstanding and out-of-order traffic line up correctly.

```
// one analysis-imp per address / write-data channel (write_*( ) callback):
`uvm_analysis_imp_decl(_master_read_address)  `uvm_analysis_imp_decl(_master_write_address)
`uvm_analysis_imp_decl(_master_write_data)     `uvm_analysis_imp_decl(_master_read_data)
`uvm_analysis_imp_decl(_slave_read_address)    `uvm_analysis_imp_decl(_slave_write_address)
`uvm_analysis_imp_decl(_slave_write_data)      `uvm_analysis_imp_decl(_slave_read_data)

// master endpoints (slave endpoints identical, typed to axi4_slave_tx):
uvm_analysis_imp_master_read_address #(axi4_master_tx,axi4_scoreboard)
  axi4_master_read_address_analysis_fifo;
uvm_tlm_analysis_fifo #(axi4_master_tx)
  axi4_master_read_data_analysis_fifo;
uvm_analysis_imp_master_write_address #(axi4_master_tx,axi4_scoreboard)
  axi4_master_write_address_analysis_fifo;
uvm_analysis_imp_master_write_data    #(axi4_master_tx,axi4_scoreboard)
  axi4_master_write_data_analysis_fifo;
uvm_tlm_analysis_fifo #(axi4_master_tx)
  axi4_master_write_response_analysis_fifo;
```

3.2.12 AXI Virtual Sequencer

Holds handles to the master and slave sequencers so a virtual sequence can coordinate both sides for a given operating mode (standalone master, standalone slave, back-to-back).

3.2.13 AXI Master Agent

Builds the master sequencer, master driver proxy and master monitor proxy, and applies the master agent configuration.

3.2.14 AXI Master Sequencer

Standard UVM sequencer arbitrating master sequences onto the master driver proxy.

3.2.15 AXI Master Driver Proxy

Branches on `transfer_type`. Non-outstanding transfers complete one at a time; outstanding transfers are buffered in dedicated TLM FIFOs so several requests can be in flight, and responses are returned to the sequencer with proper identity via `set_id_info()` / `put_response()`:

```
uvm_tlm_fifo #(axi4_master_tx) axi4_master_write_fifo_h;  
uvm_tlm_fifo #(axi4_master_tx) axi4_master_write_resp_fifo_h;  
uvm_tlm_fifo #(axi4_master_tx) axi4_master_read_fifo_h;  
  
axi4_master_write_fifo_h      = new("axi4_master_write_fifo_h",    this, 1600);  
axi4_master_write_resp_fifo_h = new("axi4_master_write_resp_fifo_h", this, 1600);  
axi4_master_read_fifo_h      = new("axi4_master_read_fifo_h",      this, 1600);
```

Read stimulus is kept read-after-write safe by `axi4_master_tx`: queues track in-flight write regions (`avoid_base/avoid_span`) and committed writes (`readback_base`), and the `read_addr_c` constraint places reads to avoid in-flight writes or to read back a written location (`read_from_written`).

```
function void axi4_master_agent::build_phase(uvm_phase phase);  
    super.build_phase(phase);  
  
    // if(!uvm_config_db #(axi4_master_agent_config)::get(this,"", "axi4_master_agent_config", axi4_master_agent_cfg_h)) begin  
    //     uvm_fatal("FATAL_MA_CANNOT_GET_MASTER_AGENT_CONFIG", "cannot get axi4_master_agent_cfg_h from uvm_config_db");  
    // end  
  
    if(axi4_master_agent_cfg_h.is_active == UVM_ACTIVE) begin  
        axi4_master_drv_proxy_h = axi4_master_driver_proxy::type_id::create("axi4_master_drv_proxy_h", this);  
        axi4_master_write_seqr_h = axi4_master_write_sequencer::type_id::create("axi4_master_write_seqr_h", this);  
        axi4_master_read_seqr_h = axi4_master_read_sequencer::type_id::create("axi4_master_read_seqr_h", this);  
    end  
  
    axi4_master_mon_proxy_h = axi4_master_monitor_proxy::type_id::create("axi4_master_mon_proxy_h", this);  
  
    if(axi4_master_agent_cfg_h.has_coverage) begin  
        axi4_master_cov_h = axi4_master_coverage::type_id::create("axi4_master_cov_h", this);  
    end  
  
endfunction : build_phase
```

Fig. 3.29 Master driver proxy run-phase

3.2.16 AXI Master Monitor Proxy

Publishes completed transfers on five analysis ports (one per AXI channel), connected to the scoreboard by the environment:

```
uvm_analysis_port #(axi4_master_tx) axi4_master_write_address_analysis_port;  
uvm_analysis_port #(axi4_master_tx) axi4_master_write_data_analysis_port;  
uvm_analysis_port #(axi4_master_tx) axi4_master_write_response_analysis_port;  
uvm_analysis_port #(axi4_master_tx) axi4_master_read_address_analysis_port;  
uvm_analysis_port #(axi4_master_tx) axi4_master_read_data_analysis_port;
```

3.2.17 AXI Slave Agent

Builds the slave sequencer, slave driver proxy and slave monitor proxy, and applies the slave agent configuration (including `slave_response_mode`).

3.2.18 AXI Slave Sequencer

Standard UVM sequencer arbitrating slave sequences onto the slave driver proxy.

3.2.19 AXI Slave Driver Proxy

Generates responses and supports both orderings selected by `slave_response_mode` (`RESP_IN_ORDER` / `RESP_OUT_OF_ORDER`). It buffers requests in per-channel TLM FIFOs and address/ID/data queues so BIDs and RIDs can be issued in or out of order:

```
uvm_tlm_fifo #(axi4_slave_tx) axi4_slave_write_addr_fifo_h;
uvm_tlm_fifo #(axi4_slave_tx) axi4_slave_write_data_in_fifo_h;
uvm_tlm_fifo #(axi4_slave_tx) axi4_slave_write_response_fifo_h;
uvm_tlm_fifo #(axi4_slave_tx) axi4_slave_write_data_out_fifo_h;
uvm_tlm_fifo #(axi4_slave_tx) axi4_slave_read_addr_fifo_h;
uvm_tlm_fifo #(axi4_slave_tx) axi4_slave_read_data_in_fifo_h;

axi4_slave_tx axiSlaveAddressQueue[$];      int axiSlaveIdQueue[$];
axi4_slave_tx axiSlaveDataQueue[$];          int axiReadSlaveIdQueue[$];
axi4_slave_tx axiReadSlaveAddressQueue[$];    bit[3:0] response_id_queue[$];
```

```
function void axi4_slave_agent::build_phase(uvm_phase phase);
    super.build_phase(phase);

    // if(!uvm_config_db #(axi4_slave_agent_config)::get(this,"", "axi4_slave_agent_config", axi4_slave_agent_cfg_h)) begin
    //     `uvm_fatal("FATAL_SA_AGENT_CONFIG", $sformatf("Couldn't get the axi4_slave_agent_config from config_db"))
    // end

    if(axi4_slave_agent_cfg_h.is_active == UVM_ACTIVE) begin
        axi4_slave_drv_proxy_h = axi4_slave_driver_proxy::type_id::create("axi4_slave_drv_proxy_h", this);
        axi4_slave_write_seqr_h = axi4_slave_write_sequencer::type_id::create("axi4_slave_write_seqr_h", this);
        axi4_slave_read_seqr_h = axi4_slave_read_sequencer::type_id::create("axi4_slave_read_seqr_h", this);
    end

    axi4_slave_mon_proxy_h = axi4_slave_monitor_proxy::type_id::create("axi4_slave_mon_proxy_h", this);

    if(axi4_slave_agent_cfg_h.has_coverage) begin
        axi4_slave_cov_h = axi4_slave_coverage::type_id::create("axi4_slave_cov_h", this);
    end
endfunction : build_phase
```

Fig. 3.37 Slave driver proxy write flow

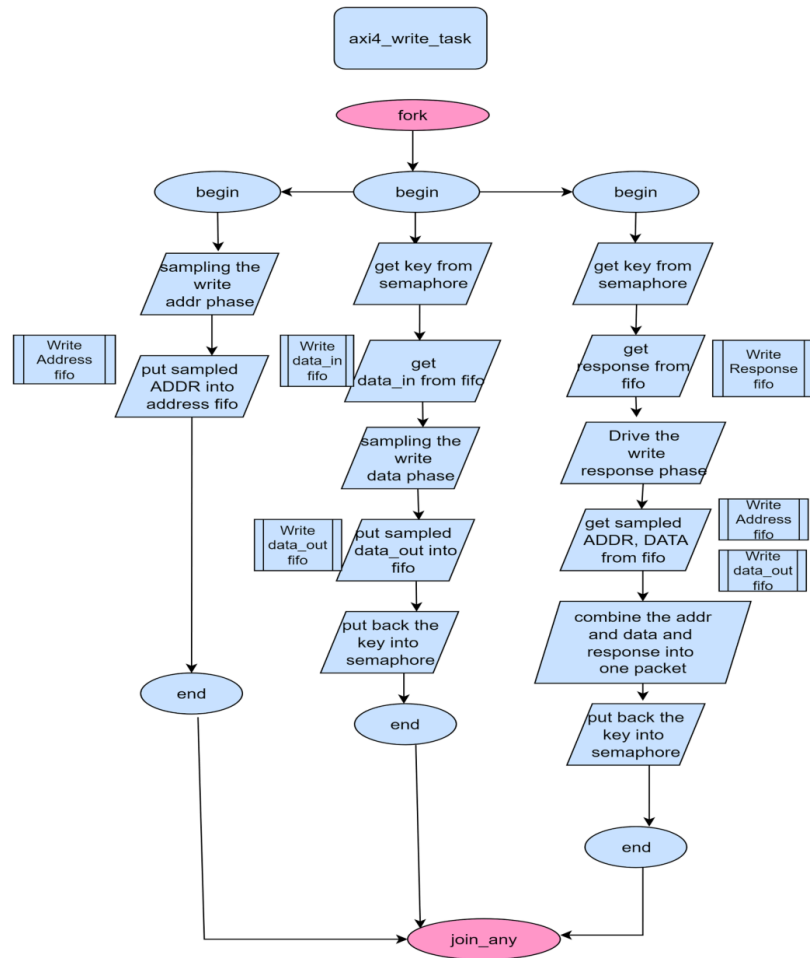


Fig. 3.39 Slave driver proxy read flow

3.2.20 AXI Slave Monitor Proxy

Publishes completed slave transfers on the same five analysis ports (typed to `axi4_slave_tx`).

3.2.21 UVM Verbosity

Simulation verbosity is controlled through the standard UVM levels (UVM_LOW / UVM_MEDIUM / UVM_HIGH / UVM_FULL) passed on the command line.

Chapter 4

Directory Structure

4.1 Package Content

The package structure guides the user to the location of each file — which folder holds it and what it contains — as shown in Fig. 4.1. Since the previous release the test environment has been reorganised around three independent **test-package families** that share a common base: **standalone master**, **standalone slave** and **back-to-back**. Each family has its own test package, virtual-sequence package and regression testlist, and the compile filelist selects one family at compile time.

```

AXI4_AVIP
├── doc/
│   ├── axi4_avip_architecture_document.pdf
│   ├── axi4_avip_user_guide.pdf
│   ├── axi4_avip_assertion_plan.md
│   ├── axi4_avip_coverage_plan.md
│   ├── standalone_master/ axi4_avip_standalone_master_verification_plan.md
│   ├── standalone_slave/  axi4_avip_standalone_slave_verification_plan.md
│   └── back_to_back/       axi4_avip_back_to_back_verification_plan.md
├── sim/
│   ├── axi4_compile.f      (MODE-selectable compile filelist)
│   ├── questasim/         (makefile: compile MODE=)
│   └── cadence_sim/
├── src/
│   ├── globals/           axi4_globals_pkg.sv
│   ├── hdl_top/           hdl_top.sv, axi4_interface/, *_agent_bfm/, assertions
│   └── hvl_top/
│       ├── master/ slave/ env/
│       └── test/
│           ├── axi4_test_base_pkg.sv          (base test)
│           ├── standalone_master/             (master test package)
│           ├── standalone_slave/              (slave test package)
│           ├── back_to_back/                  (back-to-back test package)
│           ├── sequences/master_sequences/    (axi4_master_base_seq)
│           ├── sequences/slave_sequences/     (slave base sequence)
│           ├── virtual_sequences/             (base vseq pkg)
│           │   ├── standalone_master/
│           │   ├── standalone_slave/
│           │   └── back_to_back/
│           └── testlists/                     (per-mode regression lists)

```

Fig. 4.1 Package Structure of AXI4_AVIP

Table 4.1 lists the directory paths and their contents.

Directory	Description
axi4_avip/doc	Testbench architecture and component descriptions, plus the per-mode verification plans (standalone_master, standalone_slave, back_to_back), assertion plan and coverage plan.
axi4_avip/sim	Simulator run areas (questasim, cadence_sim) and axi4_compile.f, the mode-selectable compile filelist.
axi4_avip/src/globals	Global package parameters (names, modes, widths).
axi4_avip/src/hdl_top	BFM modules, AXI4 interface and assertion files (synthesizable side).
axi4_avip/src/hdl_top/master_agent_bfm	Master agent, driver and monitor BFM files.
axi4_avip/src/hdl_top/slave_agent_bfm	Slave agent, driver and monitor BFM files.
axi4_avip/src/hdl_top/axi4_interface	AXI4 interface file.
axi4_avip/src/hvl_top	All UVM TB components (master, slave, env, test).
axi4_avip/src/hvl_top/master	Master agent files and master coverage.
axi4_avip/src/hvl_top/slave	Slave agent files and slave coverage.
axi4_avip/src/hvl_top/env	Environment configuration and scoreboard; env/virtual_sequencer.
axi4_avip/src/hvl_top/test	Base test package (axi4_test_base_pkg.sv) and the three test-package families.
axi4_avip/src/hvl_top/test/standalone_master	Standalone-master test package (master drives, slave passive).
axi4_avip/src/hvl_top/test/standalone_slave	Standalone-slave test package (slave drives responses, master passive).
axi4_avip/src/hvl_top/test/back_to_back	Back-to-back test package (active master ↔ active slave).
axi4_avip/src/hvl_top/test/sequences/master_sequences	Master base sequence and master sequence package.
axi4_avip/src/hvl_top/test/sequences/slave_sequences	Slave base sequence and slave sequence package.
axi4_avip/src/hvl_top/test/virtual_sequences	Base virtual-sequence package and per-mode virtual sequences (standalone_master, standalone_slave, back_to_back).
axi4_avip/src/hvl_top/testlists	Per-mode regression testlists (standalone_master, standalone_slave, back_to_back).

Table 4.1 Directory Path

Note: the earlier single, flat test/sequence layout has been replaced. Master and slave sequences are now reduced to a base sequence per side, and per-transfer behaviour is driven from the mode-specific virtual sequences and configuration rather than from a large flat sequence library.

Chapter 5

Configuration

5.1 Global package variables

The global variables declared in the global package file are given in this chapter.

Table 5.1 Global package variables

Name	Type	Description
awburst_e	bit	Used to declare the enum type of write burst type WRITE_FIXED = 2'b00 WRITE_INCR = 2'b01 WRITE_WRAP = 2'b10 WRITE_RESERVED = 2'b11
arburst_e	bit	Used to declare the enum type of read burst type READ_FIXED = 2'b00 READ_INCR = 2'b01 READ_WRAP = 2'b10 READ_RESERVED = 2'b11
awsiz_e	bit	Used to declare the enum type for write transfer size WRITE_1_BYTE = 3'b000 WRITE_2_BYTES = 3'b001 WRITE_4_BYTES = 3'b010 WRITE_8_BYTES = 3'b011 WRITE_16_BYTES = 3'b100 WRITE_32_BYTES = 3'b101 WRITE_64_BYTES = 3'b110 WRITE_128_BYTES = 3'b111
arsiz_e	bit	Used to declare the enum type for read transfer size READ_1_BYTE = 3'b000 READ_2_BYTES = 3'b001 READ_4_BYTES = 3'b010 READ_8_BYTES = 3'b011 READ_16_BYTES = 3'b100 READ_32_BYTES = 3'b101 READ_64_BYTES = 3'b110 READ_128_BYTES = 3'b111
awlock_e	bit	Used to declare the enum type for write lock access WRITE_NORMAL_ACCESS = 1'b0 WRITE_EXCLUSIVE_ACCESS = 1'b1
arlock_e	bit	Used to declare the enum type for read lock access READ_NORMAL_ACCESS = 1'b0 READ_EXCLUSIVE_ACCESS = 1'b1
awcache_e	bit	Used to declare the enum type for write cache access WRITE_BUFFERABLE WRITE_MODIFIABLE WRITE_OTHER_ALLOCATE WRITE_ALLOCATE
arcache_e	bit	Used to declare the enum type for read cache access READ_BUFFERABLE READ_MODIFIABLE READ_OTHER_ALLOCATE READ_ALLOCATE

awprot_e	bit	Used to represent the protection type for transaction WRITE_NORMAL_SECURE_DATA = 3'b000 WRITE_NORMAL_SECURE_INSTRUCTION = 3'b001 WRITE_NORMAL_NONSECURE_DATA = 3'b010 WRITE_NORMAL_NONSECURE_INSTRUCTION = 3'b011 WRITE_PRIVILEGED_SECURE_DATA = 3'b100 WRITE_PRIVILEGED_SECURE_INSTRUCTION = 3'b101
----------	-----	--

arprot_e	bit	Used to represent the protection type for transaction READ_NORMAL_SECURE_DATA = 3'b000 READ_NORMAL_SECURE_INSTRUCTION = 3'b001 READ_NORMAL_NONSECURE_DATA = 3'b010 READ_NORMAL_NONSECURE_INSTRUCTION = 3'b011 READ_PRIVILEGED_SECURE_DATA = 3'b100 READ_PRIVILEGED_SECURE_INSTRUCTION = 3'b101
bresp_e	bit	Represents slave error signals for write channel WRITE_OKAY = 2'b00 WRITE_EXOKAY = 2'b01 WRITE_SLVERR = 2'b10 WRITE_DECERR = 2'b11
resp_e	bit	Represents slave error signals for read channel READ_OKAY = 2'b00 READ_EXOKAY = 2'b01 READ_SLVERR = 2'b10 READ_DECERR = 2'b11
endian_e	bit	LITTLE_ENDIAN = 1'b0 : lsb bit will store in first address location BIG_ENDIAN = 1'b1 : msb bit will store in first address location
tx_type_e	bit	WRITE = 1'b1 : write transfer happens READ = 1'b0 : read transfer happens
transfer_type_e	bit	Used to determine the transfer type BLOCKING_WRITE = 2'b00 BLOCKING_READ = 2'b01 NON_BLOCKING_WRITE = 2'b10 NON_BLOCKING_READ = 2'b11

Configuration used

1. Env configuration
2. Master Agent configuration
3. Slave Agent configuration

5.2 Master agent configuration

Table 5.2 Master_agent_config

Name	Type	Default value	Description
is_active	enum	UVM_ACTIVE	It will be used for configuring an agent as an active agent means it has sequencer, driver and monitor or passive agent which has monitor only.
has_coverage	integer	'd1	Used for enabling the master agent coverage

5.3 Slave agent configuration

Table 5.3 Slave_agent_config

Name	Type	Default value	Description
is_active	enum	UVM_ACTIVE	It will be used for configuring agent as an active agent means it has sequencer, driver and monitor and if it's a passive agent then it will have only monitor
has_coverage	integer	'd1	Used for enabling the slave agent coverage.

5.4 Environment configuration

Table 5.4 Env_config

Name	Type	Default value	Description
has_scoreboard	integer	1	Enables the scoreboard, it usually receives the transaction level objects via TLM ANALYSIS PORT.
has_virtual_sqr	integer	1	Enables the virtual sequencer which has master and slave sequencer
no_of_slaves	integer	'h1	Number of slaves connected to the axi interface
no_of_masters	integer	'h1	Number of masters connected to the axi interface

5.5 Memory Mapping

Memory-mapping is a mechanism that maps a portion of a file, or an entire file, on disk to a range of addresses within an application's address space.

In AXI, the memory mapping means that the slave's address ranges are stored in the master's associative arrays so that master has access to each slave's address range.

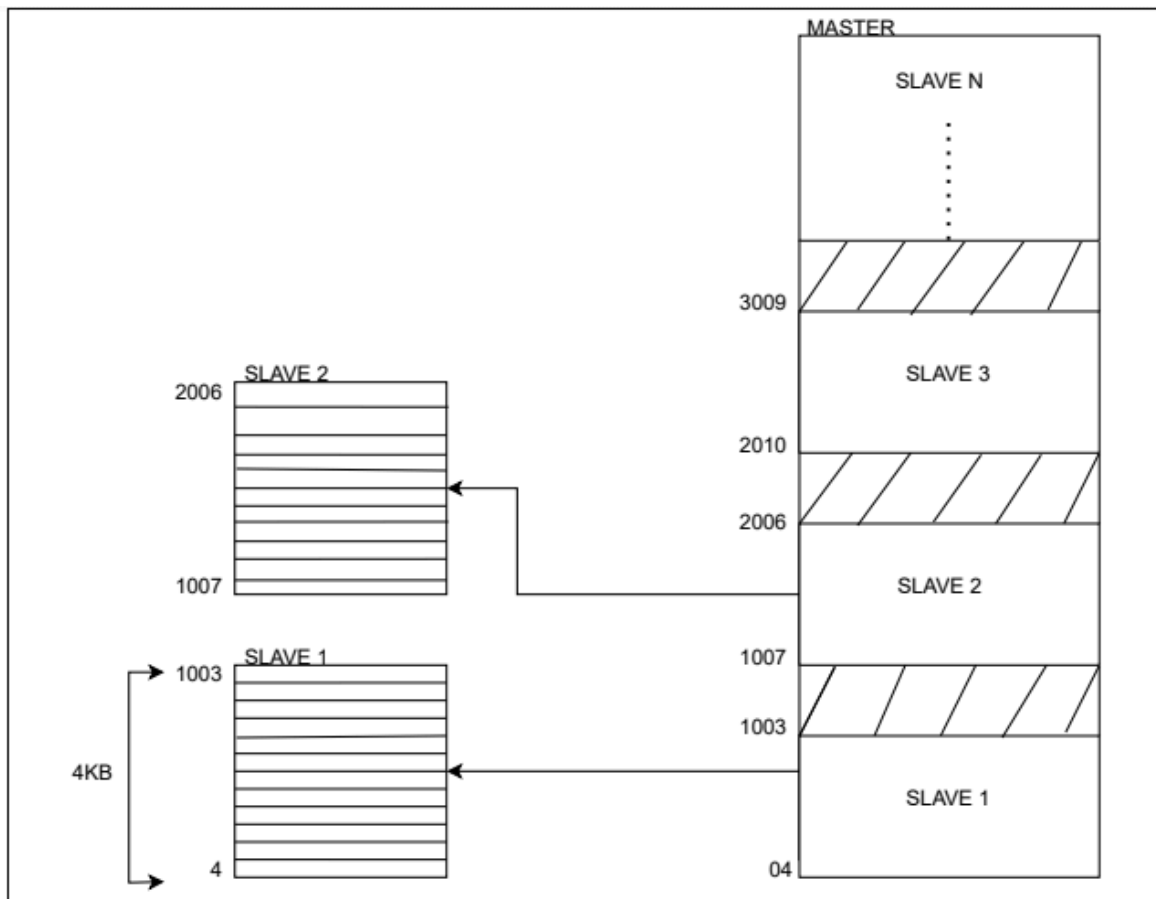


Fig. 5.5.1: Memory mapping example

Memory mapping in AXI

An example of memory mapping is as shown in fig. 1. Initially in global package, the memory is taken as **4KB**, i.e., the slave memory size is taken as **12**, because $(2^{**}\text{ADDRESS_DEPTH} = \text{MEMORY_SIZE})$ i.e., $(2^{**}12 = 4096)$ as shown in fig. 5.5.2.

Each Slave memory is given a gap of 2 locations, so that each memory mapping can be differentiated easily as shown in fig. 5.5.2.

```
//Parameter: SLAVE_MEMORY_SIZE
//Sets the memory size of the slave in KB
parameter int SLAVE_MEMORY_SIZE = 12;

//Parameter: SLAVE_MEMORY_GAP
//Sets the memory gap size of the slave
parameter int SLAVE_MEMORY_GAP = 2;
```

Fig. 5.5.2: Global parameter declaration

An associative array is used to store the *max and min address ranges* of every slave in master agent configuration, where [int] is the index type as shown in fig. 5.5.3.

```
//Variable : master_memory
//Used to store all the data from the slaves
//Each location of the master memory stores 32 bit data
bit [MEMORY_WIDTH-1:0]master_memory[(SLAVE_MEMORY_SIZE+SLAVE_MEMORY_GAP)*NO_OF_SLAVES:0];

//Variable : master_min_array
//An associative array used to store the min address ranges of every slave
//Index - type      - int
//          stores  - slave number
//Value - stores the minimum address range of that slave.
bit [ADDRESS_WIDTH-1:0]master_min_addr_range_array[int];

//Variable : master_max_array
//An associative array used to store the max address ranges of every slave
//Index - type      - int
//          stores  - slave number
//Value - stores the maximum address range of that slave.
bit [ADDRESS_WIDTH-1:0]master_max_addr_range_array[int];
```

Fig.5.5.3: Associative array declaration

In master agent configuration, two functions are written so that the value obtained will be stored in the master array as shown in fig. 5.5.4.

```
function void axi4_master_agent_config::master_max_addr_range(int slave_number, bit[ADDRESS_WIDTH-1:0]slave_max_address_range);
    master_max_addr_range_array[slave_number] = slave_max_address_range;
endfunction : master_max_addr_range

//-----
// Function : master_min_addr_range_array
// Used to store the minimum address ranges of the slaves in the array
// Parameters :
//  slave_number      - int
//  slave_min_address_range - bit [63:0]
//
function void axi4_master_agent_config::master_min_addr_range(int slave_number, bit[ADDRESS_WIDTH-1:0]slave_min_address_range);
    master_min_addr_range_array[slave_number] = slave_min_address_range;
endfunction : master_min_addr_range
```

Fig.5.5.4: Functions for memory mapping for max and min value

The memory mapping is done in base_test as shown in fig. 5. In a setup_axi4_master_agent_config(), initially, we declare 2 local variables to store the min and max address used for each iteration as shown in fig. 5.5.5.

Fig 5.5.5 Local variable declaration in function

The function setup_axi4_master_agent_cfg will start pushing the maximum and minimum address ranges to the respective associative arrays by adding a memory gap of 4 and making sure that start address is mod of 4 as shown in fig. 5.5.6.

```

for(int i =0; i<NO_OF_SLAVES; i++) begin
  if(i == 0) begin
    axi4_env_cfg_h.axi4_master_agent_cfg_h[i].master_min_addr_range(i,0);
    local_min_address = axi4_env_cfg_h.axi4_master_agent_cfg_h[i].master_min_addr_range_array[0];
    axi4_env_cfg_h.axi4_master_agent_cfg_h[i].master_max_addr_range(i,2**(SLAVE_MEMORY_SIZE)-1 );
    local_max_address = axi4_env_cfg_h.axi4_master_agent_cfg_h[i].master_max_addr_range_array[i];
  end
  else begin
    axi4_env_cfg_h.axi4_master_agent_cfg_h[i].master_min_addr_range(i,local_max_address + SLAVE_MEMORY_GAP);
    local_min_address = axi4_env_cfg_h.axi4_master_agent_cfg_h[i].master_min_addr_range_array[i];
    axi4_env_cfg_h.axi4_master_agent_cfg_h[i].master_max_addr_range(i,local_max_address+ 2**(SLAVE_MEMORY_SIZE)-1 +
      SLAVE_MEMORY_GAP);
    local_max_address = axi4_env_cfg_h.axi4_master_agent_cfg_h[i].master_max_addr_range_array[i];
  end
  `uvm_info(get_type_name(),$formatf("\nAXI4_MASTER_CONFIG[%0d]\n%s",i,axi4_env_cfg_h.axi4_master_agent_cfg_h[i].sprint()),UVM_L
OW);
end

```

Fig 5.5.6: Memory mapping procedure in master agent configuration

In slave agent configuration, the *slave max and min address range* is declared as shown in fig. 5.5.7. Created the slave memory of type associative array so that each slave can store the data received from master with the respective address as key.

```

//Variable : max_address
//Used to store the maximum address value of this slave
bit [ADDRESS_WIDTH-1:0]max_address;

//Variable : min_address
//Used to store the minimum address value of this slave
bit [ADDRESS_WIDTH-1:0]min_address;

//Variable : slave_memory
//Declaration of slave_memory to store the data from master
bit [7:0]slave_memory[longint];

```

Fig 5.5.7: Declaration of slave max and min address range

Similarly for Slave, the mapping is done as shown in fig. 5.5.8. The same index value is mapped for the slave memory, so that the slave stores the data in the same address range for memory. Each slave's minimum and maximum addressess are sent to the respective slave agent configurations from the stored maximum and minimum address ranges in the master agent configuration.

```

//-----
// Function: setup_axi4_slave_agents_cfg
// Setup the axi4 slave agent(s) configuration with the required values
// and store the handle into the config_db
//-----
function void axi4_base_test::setup_axi4_slave_agent_cfg();
axi4_env_cfg_h.axi4_slave_agent_cfg_h = new[axi4_env_cfg_h.no_of_slaves];
foreach(axi4_env_cfg_h.axi4_slave_agent_cfg_h[i])begin
    axi4_env_cfg_h.axi4_slave_agent_cfg_h[i] =
        axi4_slave_agent_cfg::type_id::create($sformatf("axi4_slave_agent_cfg_h[%0d]",i));
    axi4_env_cfg_h.axi4_slave_agent_cfg_h[i].slave_id = i;
    //axi4_env_cfg_h.axi4_slave_agent_cfg_h[i].s_slave_selected = 0;
    axi4_env_cfg_h.axi4_slave_agent_cfg_h[i].min_address = axi4_env_cfg_h.axi4_master_agent_cfg_h[i].
        master_min_addr_range_array[1];
    axi4_env_cfg_h.axi4_slave_agent_cfg_h[i].max_address = axi4_env_cfg_h.axi4_master_agent_cfg_h[i].
        master_max_addr_range_array[1];

    if(SLAVE_AGENT_ACTIVE == 1) begin
        axi4_env_cfg_h.axi4_slave_agent_cfg_h[i].is_active = uvm_active_passive_enum'(UVM_ACTIVE);
    end
    else begin
        axi4_env_cfg_h.axi4_slave_agent_cfg_h[i].is_active = uvm_active_passive_enum'(UVM_PASSIVE);
    end
    axi4_env_cfg_h.axi4_slave_agent_cfg_h[i].has_coverage = 1;

    uvm_config_db #(axi4_slave_agent_cfg)::set(this,"env",$sformatf("axi4_slave_agent_config[%0d]",i), axi4_env_cfg_h.axi4_slave_agent_cfg_h[i]);
    'uvm_info(get_type_name(),$sformatf("\nAXI4_SLAVE_CONFIG[%0d]\nms",i,axi4_env_cfg_h.axi4_slave_agent_cfg_h[i].sprint()),UVM_LOW);
end
endfunction: setup_axi4_slave_agent_cfg

```

Fig.5.5.8: Memory mapping procedure in slave agent configuration

Chapter 6

Verification Plan

6.1 Verification plan

The verification plan captures every feature of the AXI4 protocol that must be exercised, the sub-features under each, a short description, the implementation status and the name of the test that closes it. In this release the plan is partitioned into **three per-mode verification plans**, one for each test-package family, so that each operating mode can be planned, tracked and signed off independently:

- `doc/standalone_master/axi4_avip_standalone_master_verification_plan.md`
- `doc/standalone_slave/axi4_avip_standalone_slave_verification_plan.md`
- `doc/back_to_back/axi4_avip_back_to_back_verification_plan.md`

6.2 Template of Verification Plan

Each plan uses a common template with the following columns:

- **S.No / Sections** — the operating mode and the feature grouping being verified.
- **Features / Sub-Features** — the protocol capability and its variants (data width, burst type, response type, outstanding/non-outstanding).
- **Description** — the intent of the scenario.
- **Status** — DONE / WIP / TODO.
- **Testcase Name** — the test that implements the scenario.

6.3 Sections for different test Scenarios

6.3.1 Standalone Master Verification Plan

Master drives the transfers while the slave stays passive; used to bring up and verify the master RTL/agent.

S.No	Sections	Features	Sub-Features	Description	Status	Testcase Name
1	Standalone Master	Non-Outstanding	write	Master drives writes, slave passive	DONE	axi4_non_outstanding_slave_write_test
			read	Master drives reads, slave passive	DONE	axi4_non_outstanding_slave_read_test
			write & read	Master drives write+read, slave passive	DONE	axi4_non_outstanding_slave_write_read_test
2	Standalone Master	Outstanding	write	Master drives outstanding writes	DONE	axi4_outstanding_slave_write_test
			read	Master drives outstanding reads	DONE	axi4_outstanding_slave_read_test
			write & read	Master drives outstanding write+read	DONE	axi4_outstanding_slave_write_read_test

Table 6.1 Standalone Master verification plan

6.3.2 Standalone Slave Verification Plan

Slave generates the responses across all data widths, burst types and response types while the master is passive.

S.No	Sections	Features	Sub-Features	Description	Status	Testcase Name
1	Standalone Slave	Base	write & read	Basic write+read, master passive	DONE	axi4_non_outstanding_write_read_test
2	Standalone Slave	Ordered	32b write & read	32b ordered write+read responses	DONE	axi4_32b_ordered_write_read_test
3	Standalone Slave	Response	okay write	OKAY write response	DONE	axi4_non_outstanding_okay_response_write_test
			okay read	OKAY read response	DONE	axi4_non_outstanding_okay_response_read_test
			okay write & read	OKAY write+read response	DONE	axi4_non_outstanding_okay_response_write_read_test
			exokay write	EXOKAY (exclusive) write response	DONE	axi4_non_outstanding_exokay_response_write_test
			exokay read	EXOKAY (exclusive) read response	DONE	axi4_non_outstanding_exokay_response_read_test
4	Standalone Slave	Non-Outstanding . 8 bits	write data	8b write data	DONE	axi4_non_outstanding_8b_write_data_test
			data read	8b read data	DONE	axi4_non_outstanding_8b_data_read_test
			write & read	8b write read	DONE	axi4_non_outstanding_8b_write_read_test
5	Standalone Slave	Non-Outstanding . 16 bits	write data	16b write data	DONE	axi4_non_outstanding_16b_write_data_test
			data read	16b read data	DONE	axi4_non_outstanding_16b_data_read_test
			write & read	16b write read	DONE	axi4_non_outstanding_16b_write_read_test
6	Standalone Slave	Non-Outstanding . 32 bits	write data	32b write data	DONE	axi4_non_outstanding_32b_write_data_test
			data read	32b read data	DONE	axi4_non_outstanding_32b_data_read_test
			write & read	32b write read	DONE	axi4_non_outstanding_32b_write_read_test
7	Standalone Slave	Non-Outstanding . 64 bits	write data	64b write data	DONE	axi4_non_outstanding_64b_write_data_test
			data read	64b read data	DONE	axi4_non_outstanding_64b_data_read_test
			write & read	64b write read	DONE	axi4_non_outstanding_64b_write_read_test
8	Standalone Slave	Non-Outstanding . 128 bits	write data	128b write data	DONE	axi4_non_outstanding_128b_write_data_test
			data read	128b read data	DONE	axi4_non_outstanding_128b_data_read_test
			write & read	128b write read	DONE	axi4_non_outstanding_128b_write_read_test
			fixed burst	128b fixed burst write read	DONE	axi4_non_outstanding_128b_fixed_burst_write_read_test

S.No	Sections	Features	Sub-Features	Description	Status	Testcase Name
			wrap burst	128b wrap burst write read	DONE	axi4_non_outstanding_128b_wrap_burst_write_read_test
9	Standalone Slave	Non-Outstanding - 256 bits	write data	256b write data	DONE	axi4_non_outstanding_256b_write_data_test
			data read	256b read data	DONE	axi4_non_outstanding_256b_data_read_test
			write & read	256b write read	DONE	axi4_non_outstanding_256b_write_read_test
			fixed burst	256b fixed burst write read	DONE	axi4_non_outstanding_256b_fixed_burst_write_read_test
			wrap burst	256b wrap burst write read	DONE	axi4_non_outstanding_256b_wrap_burst_write_read_test
10	Standalone Slave	Non-Outstanding - 512 bits	write data	512b write data	DONE	axi4_non_outstanding_512b_write_data_test
			data read	512b read data	DONE	axi4_non_outstanding_512b_data_read_test
			write & read	512b write read	DONE	axi4_non_outstanding_512b_write_read_test
			fixed burst	512b fixed burst write read	DONE	axi4_non_outstanding_512b_fixed_burst_write_read_test
			wrap burst	512b wrap burst write read	DONE	axi4_non_outstanding_512b_wrap_burst_write_read_test
11	Standalone Slave	Non-Outstanding - Burst	incr write	INCR burst write	DONE	axi4_non_outstanding_incr_burst_write_test
			incr read	INCR burst read	DONE	axi4_non_outstanding_incr_burst_read_test
			incr write & read	INCR burst write read	DONE	axi4_non_outstanding_incr_burst_write_read_test
			wrap write	WRAP burst write	DONE	axi4_non_outstanding_wrap_burst_write_test
			wrap read	WRAP burst read	DONE	axi4_non_outstanding_wrap_burst_read_test
			wrap write & read	WRAP burst write read	DONE	axi4_non_outstanding_wrap_burst_write_read_test
			fixed write & read	FIXED burst write read	DONE	axi4_non_outstanding_fixed_burst_write_read_test
12	Standalone Slave	Outstanding - 8 bits	write data	8b write data	DONE	axi4_outstanding_8b_write_data_test
			data read	8b read data	DONE	axi4_outstanding_8b_data_read_test
			write & read	8b write read	DONE	axi4_outstanding_8b_write_read_test
13	Standalone Slave	Outstanding - 16 bits	write data	16b write data	DONE	axi4_outstanding_16b_write_data_test
			data read	16b read data	DONE	axi4_outstanding_16b_data_read_test
			write & read	16b write read	DONE	axi4_outstanding_16b_write_read_test
14	Standalone Slave	Outstanding - 32 bits	write data	32b write data	DONE	axi4_outstanding_32b_write_data_test
			data read	32b read data	DONE	axi4_outstanding_32b_data_read_test
			write & read	32b write read	DONE	axi4_outstanding_32b_write_read_test

S.No	Sections	Features	Sub-Features	Description	Status	Testcase Name
15	Standalone Slave	Outstanding - 64 bits	write data	64b write data	DONE	axi4_outstanding_64b_write_data_test
			data read	64b read data	DONE	axi4_outstanding_64b_data_read_test
			write & read	64b write read	DONE	axi4_outstanding_64b_write_read_test
16	Standalone Slave	Outstanding - 128 bits	write data	128b write data	DONE	axi4_outstanding_128b_write_data_test
			data read	128b read data	DONE	axi4_outstanding_128b_data_read_test
			write & read	128b write read	DONE	axi4_outstanding_128b_write_read_test
			fixed burst	128b fixed burst write read	DONE	axi4_outstanding_128b_fixed_burst_write_read_test
			wrap burst	128b wrap burst write read	DONE	axi4_outstanding_128b_wrap_burst_write_read_test
17	Standalone Slave	Outstanding - 256 bits	write data	256b write data	DONE	axi4_outstanding_256b_write_data_test
			data read	256b read data	DONE	axi4_outstanding_256b_data_read_test
			write & read	256b write read	DONE	axi4_outstanding_256b_write_read_test
			fixed burst	256b fixed burst write read	DONE	axi4_outstanding_256b_fixed_burst_write_read_test
			wrap burst	256b wrap burst write read	DONE	axi4_outstanding_256b_wrap_burst_write_read_test
18	Standalone Slave	Outstanding - 512 bits	write data	512b write data	DONE	axi4_outstanding_512b_write_data_test
			data read	512b read data	DONE	axi4_outstanding_512b_data_read_test
			write & read	512b write read	DONE	axi4_outstanding_512b_write_read_test
			fixed burst	512b fixed burst write read	DONE	axi4_outstanding_512b_fixed_burst_write_read_test
			wrap burst	512b wrap burst write read	DONE	axi4_outstanding_512b_wrap_burst_write_read_test
19	Standalone Slave	Outstanding - Burst	incr write	INCR burst write	DONE	axi4_outstanding_incr_burst_write_test
			incr read	INCR burst read	DONE	axi4_outstanding_incr_burst_read_test
			incr write & read	INCR burst write read	DONE	axi4_outstanding_incr_burst_write_read_test
			wrap write	WRAP burst write	DONE	axi4_outstanding_wrap_burst_write_test
			wrap read	WRAP burst read	DONE	axi4_outstanding_wrap_burst_read_test
			wrap write & read	WRAP burst write read	DONE	axi4_outstanding_wrap_burst_write_read_test
			fixed write & read	FIXED burst write read	DONE	axi4_outstanding_fixed_burst_write_read_test

Table 6.2 Standalone Slave verification plan

6.3.3 Back-to-Back Verification Plan

Active master and active slave run together, covering non-outstanding and outstanding transfers back to back across all data widths and burst types.

S.No	Sections	Features	Sub-Features	Description	Status	Testcase Name
1	Back-to-Back	Base	write	Back-to-back write transfers	DONE	axi4_back_to_back_write_test
			read	Back-to-back read transfers	DONE	axi4_back_to_back_read_test
			write & read	Back-to-back write and read transfers	DONE	axi4_back_to_back_write_read_test
2	Back-to-Back	Non-Outstanding · 8 bits	write data	8b write data	DONE	axi4_back_to_back_non_outstanding_8b_write_data_test
			data read	8b read data	DONE	axi4_back_to_back_non_outstanding_8b_data_read_test
			write & read	8b write read	DONE	axi4_back_to_back_non_outstanding_8b_write_read_test
3	Back-to-Back	Non-Outstanding · 16 bits	write data	16b write data	DONE	axi4_back_to_back_non_outstanding_16b_write_data_test
			data read	16b read data	DONE	axi4_back_to_back_non_outstanding_16b_data_read_test
			write & read	16b write read	DONE	axi4_back_to_back_non_outstanding_16b_write_read_test
4	Back-to-Back	Non-Outstanding · 32 bits	write data	32b write data	DONE	axi4_back_to_back_non_outstanding_32b_write_data_test
			data read	32b read data	DONE	axi4_back_to_back_non_outstanding_32b_data_read_test
			write & read	32b write read	DONE	axi4_back_to_back_non_outstanding_32b_write_read_test
5	Back-to-Back	Non-Outstanding · 64 bits	write data	64b write data	DONE	axi4_back_to_back_non_outstanding_64b_write_data_test
			data read	64b read data	DONE	axi4_back_to_back_non_outstanding_64b_data_read_test
			write & read	64b write read	DONE	axi4_back_to_back_non_outstanding_64b_write_read_test
6	Back-to-Back	Non-Outstanding · 128 bits	write data	128b write data	DONE	axi4_back_to_back_non_outstanding_128b_write_data_test
			data read	128b read data	DONE	axi4_back_to_back_non_outstanding_128b_data_read_test
			write & read	128b write read	DONE	axi4_back_to_back_non_outstanding_128b_write_read_test
			fixed burst	128b fixed burst write read	DONE	axi4_back_to_back_non_outstanding_128b_fixed_burst_write_read_test
			wrap burst	128b wrap burst write read	DONE	axi4_back_to_back_non_outstanding_128b_wrap_burst_write_read_test

S.No	Sections	Features	Sub-Features	Description	Status	Testcase Name
7	Back-to-Back	Non-Outstanding · 256 bits	write data	256b write data	DONE	axi4_back_to_back_non_outstanding_256b_write_data_test
			data read	256b read data	DONE	axi4_back_to_back_non_outstanding_256b_data_read_test
			write & read	256b write read	DONE	axi4_back_to_back_non_outstanding_256b_write_read_test
			fixed burst	256b fixed burst write read	DONE	axi4_back_to_back_non_outstanding_256b_fixed_burst_write_read_test
			wrap burst	256b wrap burst write read	DONE	axi4_back_to_back_non_outstanding_256b_wrap_burst_write_read_test
8	Back-to-Back	Non-Outstanding · 512 bits	write data	512b write data	DONE	axi4_back_to_back_non_outstanding_512b_write_data_test
			data read	512b read data	DONE	axi4_back_to_back_non_outstanding_512b_data_read_test
			write & read	512b write read	DONE	axi4_back_to_back_non_outstanding_512b_write_read_test
			fixed burst	512b fixed burst write read	DONE	axi4_back_to_back_non_outstanding_512b_fixed_burst_write_read_test
			wrap burst	512b wrap burst write read	DONE	axi4_back_to_back_non_outstanding_512b_wrap_burst_write_read_test
9	Back-to-Back	Non-Outstanding · Burst	incr write	INCR burst write	DONE	axi4_back_to_back_non_outstanding_incr_burst_write_test
			incr read	INCR burst read	DONE	axi4_back_to_back_non_outstanding_incr_burst_read_test
			incr write & read	INCR burst write read	DONE	axi4_back_to_back_non_outstanding_incr_burst_write_read_test
			wrap write	WRAP burst write	DONE	axi4_back_to_back_non_outstanding_wrap_burst_write_test
			wrap read	WRAP burst read	DONE	axi4_back_to_back_non_outstanding_wrap_burst_read_test
			wrap write & read	WRAP burst write read	DONE	axi4_back_to_back_non_outstanding_wrap_burst_write_read_test
			fixed write & read	FIXED burst write read	DONE	axi4_back_to_back_non_outstanding_fixed_burst_write_read_test
10	Back-to-Back	Outstanding · 8 bits	write data	8b write data	DONE	axi4_back_to_back_outstanding_8b_write_data_test
			data read	8b read data	DONE	axi4_back_to_back_outstanding_8b_data_read_test
			write & read	8b write read	DONE	axi4_back_to_back_outstanding_8b_write_read_test
11	Back-to-Back	Outstanding · 16 bits	write data	16b write data	DONE	axi4_back_to_back_outstanding_16b_write_data_test
			data read	16b read data	DONE	axi4_back_to_back_outstanding_16b_data_read_test

S.No	Sections	Features	Sub-Features	Description	Status	Testcase Name
			write & read	16b write read	DONE	axi4_back_to_back_outstanding_16b_write_read_test
12	Back-to-Back	Outstanding · 32 bits	write data	32b write data	DONE	axi4_back_to_back_outstanding_32b_write_data_test
			data read	32b read data	DONE	axi4_back_to_back_outstanding_32b_data_read_test
			write & read	32b write read	DONE	axi4_back_to_back_outstanding_32b_write_read_test
13	Back-to-Back	Outstanding · 64 bits	write data	64b write data	DONE	axi4_back_to_back_outstanding_64b_write_data_test
			data read	64b read data	DONE	axi4_back_to_back_outstanding_64b_data_read_test
			write & read	64b write read	DONE	axi4_back_to_back_outstanding_64b_write_read_test
14	Back-to-Back	Outstanding · 128 bits	write data	128b write data	DONE	axi4_back_to_back_outstanding_128b_write_data_test
			data read	128b read data	DONE	axi4_back_to_back_outstanding_128b_data_read_test
			write & read	128b write read	DONE	axi4_back_to_back_outstanding_128b_write_read_test
			fixed burst	128b fixed burst write read	DONE	axi4_back_to_back_outstanding_128b_fixed_burst_write_read_test
			wrap burst	128b wrap burst write read	DONE	axi4_back_to_back_outstanding_128b_wrap_burst_write_read_test
15	Back-to-Back	Outstanding · 256 bits	write data	256b write data	DONE	axi4_back_to_back_outstanding_256b_write_data_test
			data read	256b read data	DONE	axi4_back_to_back_outstanding_256b_data_read_test
			write & read	256b write read	DONE	axi4_back_to_back_outstanding_256b_write_read_test
			fixed burst	256b fixed burst write read	DONE	axi4_back_to_back_outstanding_256b_fixed_burst_write_read_test
			wrap burst	256b wrap burst write read	DONE	axi4_back_to_back_outstanding_256b_wrap_burst_write_read_test
16	Back-to-Back	Outstanding · 512 bits	write data	512b write data	DONE	axi4_back_to_back_outstanding_512b_write_data_test
			data read	512b read data	DONE	axi4_back_to_back_outstanding_512b_data_read_test
			write & read	512b write read	DONE	axi4_back_to_back_outstanding_512b_write_read_test
			fixed burst	512b fixed burst write read	DONE	axi4_back_to_back_outstanding_512b_fixed_burst_write_read_test
			wrap burst	512b wrap burst write read	DONE	axi4_back_to_back_outstanding_512b_wrap_burst_write_read_test
17	Back-to-Back	Outstanding · Burst	incr write	INCR burst write	DONE	axi4_back_to_back_outstanding_incr_burst_write_test

S.No	Sections	Features	Sub-Features	Description	Status	Testcase Name
			incr read	INCR burst read	DONE	axi4_back_to_back_outstanding_incr_burst_read_test
			incr write & read	INCR burst write read	DONE	axi4_back_to_back_outstanding_incr_burst_write_read_test
			wrap write	WRAP burst write	DONE	axi4_back_to_back_outstanding_wrap_burst_write_test
			wrap read	WRAP burst read	DONE	axi4_back_to_back_outstanding_wrap_burst_read_test
			wrap write & read	WRAP burst write read	DONE	axi4_back_to_back_outstanding_wrap_burst_write_read_test
			fixed write & read	FIXED burst write read	DONE	axi4_back_to_back_outstanding_fixed_burst_write_read_test

Table 6.3 Back-to-Back verification plan

Chapter 7

Assertion Plan

7.1 Assertion Plan overview

Assertion plan is an important step in verification flow, which validates the behaviour of design at every instance.

7.1.1 What are assertions?

- An assertion specifies the behaviour of the system.
- Piece of verification code that monitors a design implementation for compliance with the specifications
- Directive to a verification tool that the tool should attempt to prove/assume/count a given property using formal methods

7.1.2 Why do we use it?

- Assertions are primarily used to validate the behaviour of a design.
- Assertions can be used to provide functional coverage and to flag that input stimulus, which is used for validation, does not conform to assumed requirements.
- Assertions are used to find more bugs and source the bugs faster.

7.1.3 Benefits of Assertions

- Improves observability of the design.
- Improves debugging of the design.
- Improves documentation of the design.

7.2 Template of Assertion Plan

Template for Assertion plan is done in an excel sheet and refer to link below:

 [axi4_avip_assertion_plan](#)

7.3 Assertion Condition

```
//-----  
// Assertion properties written for various checks in write address channel  
//-----  
//Assertion: AXI_WA_STABLE_SIGNALS_CHECK  
//Description: All signals must remain stable after AWVALID is asserted until AWREADY IS LOW  
property if_write_address_channel_signals_are_stable;  
  @(posedge aclk) disable iff (!aresetn)  
    (awvalid==1 && awready==0) |> ($stable(awid) && $stable(awaddr) && $stable(awlen) && $stable(awsize) &&  
                                   $stable(awburst) && $stable(awlock) && $stable(awcache) && $stable(awprot));  
endproperty : if_write_address_channel_signals_are_stable  
AXI_WA_STABLE_SIGNALS_CHECK: assert property (if_write_address_channel_signals_are_stable);
```

Fig. 7.1 Assertion code for stable signals check

Property AXI_WA_STABLE_SIGNALS_CHECK is evaluated as follows:

- Initially it will check for posedge of aclk and aresetn should be high.
- When awvalid is high and awready is low ,at the same cycle it will check for awid,awaddr,awlen,awsize,awburst,awlock,awcache and awprot signal should be stable. Then property is true.
- Otherwise the property will fail.

7.3.2. AXI_WA_UNKNOWN_SIGNALS_CHECK

A value of X on signals is not permitted when AWVALID is high as shown in fig. 7.2

```
//Assertion: AXI_WA_UNKNOWN_SIGNALS_CHECK  
//Description: A value of X on signals is not permitted when AWVALID is HIGH  
property if_write_address_channel_signals_are_unknown;  
  @(posedge aclk) disable iff (!aresetn)  
    (awvalid==1) |-> (!($isunknown(awid)) && !($isunknown(awaddr)) && !($isunknown(awlen)) && !($isunknown(awsize))  
                    && !($isunknown(awburst)) && !($isunknown(awlock)) && !($isunknown(awcache)) && !($isunknown(awprot)));  
endproperty : if_write_address_channel_signals_are_unknown  
AXI_WA_UNKNOWN_SIGNALS_CHECK: assert property (if_write_address_channel_signals_are_unknown);
```

Fig. 7.2 Assertion code for unknown signals check

Property AXI_WA_UNKNOWN_SIGNALS_CHECK is evaluated as follows:

- Initially it will check for posedge of aclk and aresetn should be high.
- When awvalid is high,at the same cycle it will check for awid,awaddr,awlen,awsize,awburst,awlock,awcache and awprot signal should be unknown. Then property is true.
- Otherwise the property will fail.

7.3.3. AXI_WA_VALID_STABLE_CHECK

When **AWVALID** is asserted, then it must remain asserted until **AWREADY** is high as shown in fig. 7.3

Fig. 7.3 Assertion code for valid stable check

```
//Assertion:  AXI_WA_VALID_STABLE_CHECK
//Description: When AWVALID is asserted, then it must remain asserted until AWREADY is HIGH
//Assertion stays asserted from the time awvalid becomes high and till awready becomes high using s_until_with keyword
property axi_write_address_channel_valid_stable_check;
  @(posedge aclk) disable iff (!aresetn)
  $rose(awvalid) |-> awvalid s_until_with awready;
endproperty : axi_write_address_channel_valid_stable_check
AXI_WA_VALID_STABLE_CHECK : assert property (axi_write_address_channel_valid_stable_check);
```

Property AXI_WA_VALID_STABLE_CHECK is evaluated as follows:

- Initially it will check for posedge of aclk and aresetn should be high.
- When awvalid is changed from zero to high, at the same cycle it will check for awvalid should be high until awready signal is high. Then property is true.
- Otherwise the property will fail.

Chapter 8

Coverage

8.1 Functional Coverage

- Functional coverage is the coverage data generated from the user defined functional coverage model and assertions usually written in System Verilog. During simulation, the simulator generates functional coverage based on the stimulus. Looking at the functional coverage data, one can identify the portions of the DUT [Features] verified. Also, it helps us to target the DUT features that are unverified.
- The reason for switching to the functional coverage is that we can create the bins manually as per our requirement while in the code coverage it is generated by the system by itself.

8.2 Uvm_Subscriber

- This class provides an analysis export for receiving transactions from a connected analysis export. Making such a connection "subscribes" this component to any transactions emitted by the connected analysis port. Subtypes of this class must define the write method to process the incoming transactions. This class is particularly useful when designing a coverage collector that attaches to a monitor.

```
virtual class uvm_subscriber #(type T=int) extends uvm_component;
typedef uvm_subscriber #(T) this_type;

// Port: analysis_export
//
// This export provides access to the write method, which derived subscribers
// must implement.

uvm_analysis_imp #(T, this_type) analysis_export;

// Function: new
//
// Creates and initializes an instance of this class using the normal
// constructor arguments for <uvm_component>: ~name~ is the name of the
// instance, and ~parent~ is the handle to the hierarchical parent, if any.

function new (string name, uvm_component parent);
    super.new(name, parent);
    analysis_export = new("analysis_imp", this);
endfunction

// Function: write
//
// A pure virtual method that must be defined in each subclass. Access
// to this method by outside components should be done via the
// analysis_export.

pure virtual function void write(T t);

endclass
```

Figure8.1 . Uvm_subscriber

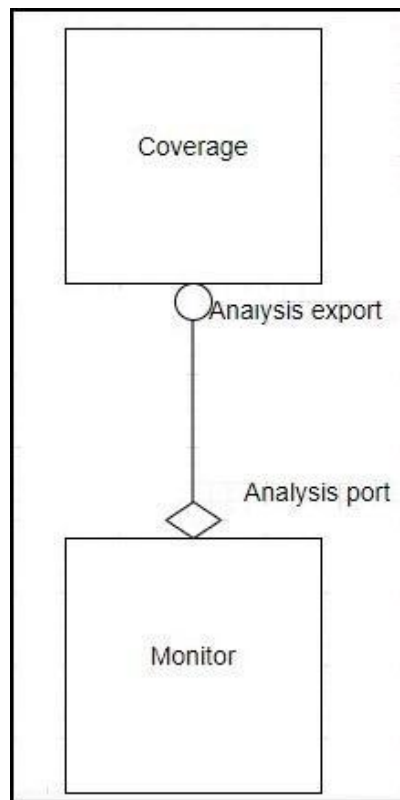


Figure8.2 . Monitor and coverage connection

8.2.1 Analysis export

This export provides access to the write method, which derived subscribers must implement.

8.2.2 Write function

The write function is to process the incoming transactions.

```

function void axi4_master_coverage::write(axi4_master_tx t);
`uvm_info(get_type_name(),$sformatf("Before calling SAMPLE METHOD"),UVM_HIGH);

axi4_master_covergroup.sample(axi4_master_agent_cfg_h,t);

`uvm_info(get_type_name(),"After calling SAMPLE METHOD",UVM_HIGH);
endfunction: write
  
```

Figure 8.3. Write function

8.3 Covergroup

```
covergroup axi4 master covergroup with function sample axi4 master agent config cfg, axi4 master_tx packet);
option.per_instance = 1;
//-----
// Write channel signals
//-----
AWLEN CP : coverpoint packet.awlen {
option.comment = "awlen";
bins AW_LEN[16]={0:$}};
}
```

Figure 8.4. Covergroup

The above red mark points in Figure 8.3 is explained below :-

1. **With function sample** - It is used to pass a variable to covergroup.
2. Parameter based on which the coverpoint is generated.
3. **Per Instance Coverage - 'option.per_instance'**

In your test bench, you might have instantiated coverage_group multiple times. By default, System Verilog collects all the coverage data from all the instances. You might have more than one generator and they might generate different streams of transaction. In this case you may want to see separate reports. Using this option, you can keep track of coverage for each instance.

3.1. option.per_instance=1 Each instance contributes to the overall coverage information for the covergroup type. When true, coverage information for this covergroup instance shall be saved in the coverage database and included in the coverage report.

```
covergroup axi4_master_covergroup with function sample (axi4_master_agent_config cfg, axi4_master_tx packet);
option.per_instance = 1;
```

Questa Covergroup Coverage Report						
Search: cvg:axi4_master_covergroup						
Covergroups/Instances	Total Bins	Hits	Misses	Hits %	Goal %	Coverage %
Covergroup axi4_master_covergroup	2336	47	2289	2.01%	22.81%	22.81%
Instance Vaxi4_master_pkg::axi4_master...erage::axi4_master_covergroup	2336	47	2289	2.01%	22.81%	22.81%

Figure 8.5. option.per_instance

4. Cover Group Comment - 'option.comment'

You can add a comment in to coverage report to make them easier while analysing:

Coverpoint: ARBURST_CP		
Comment:  arburst		
Search:		
Bin Name	At Least	Hits
ARBURST[READ_FIXED]	1	12
ARBURST[READ_INCR]	1	0
ARBURST[READ_WRAP]	1	0
ARBURST[READ_RESERVED]	1	0

Figure 8.6 option.comment

For example, you could see the usage of 'option.comment' feature. This way you can make the coverage group easier for the analysis.

8.4 Coverpoints

There we are created the bins based on the write_address_length, write_address_size, write_address_burst

```

AWLEN_CP : coverpoint packet.awlen {
    option.comment = "awlen";
    bins AW_LEN[16]={[0:$]};
}

AWBURST_CP : coverpoint packet.awburst {
    option.comment = "awburst";
    bins AWBURST[]={[0:$]};
}

AWSIZE_CP : coverpoint packet.awsize {
    option.comment = "awsize";
    bins AWSIZE[]={[0:$]};
}

```

Figure 8.7. Coverpoint

8.5 Illegal bins

```
illegal_bins illegal_bin = {0};
```

Illegal bins are used when we don't want to have the particular value eg - we don't want to have the baud_rate_divisor to be zero so we create the illegal bin for it.

8.6 Creation of the covergroup

```
function axi4_master_coverage::new(string name = "axi4_master_coverage",  
                                   uvm_component parent = null);  
    super.new(name, parent);  
    axi4_master_covergroup = new();  
endfunction : new
```

Figure 8.8. Creation of covergroup

In this function the creation of the covergroup is done with the new as shown in the figure above.

8.7 Sampling of the covergroup

In this the sampling of the covergroup is done in the write function as shown below

```
function void axi4_master_coverage::write(axi4_master_tx t);  
    `uvm_info(get_type_name(), $sformatf("Before calling SAMPLE METHOD"), UVM_HIGH);  
  
    axi4_master_covergroup.sample(axi4_master_agent_cfg_h, t);  
  
    `uvm_info(get_type_name(), "After calling SAMPLE METHOD", UVM_HIGH);  
endfunction : write
```

Figure 8.9. Sampling of the covergroup

8.8 Checking for the coverage

1. Make Compile
2. Make simulate
3. Open the log file

```
Log file path: axi4_blocking_write_read_test/axi4_blocking_write_read_test.log
```

Figure 8.10. Log file

4. Search for the coverage (There it will be the full coverage) in the log file.
5. To check the individual coverage bins hit open the coverage report as shown :-

```
Coverage report: firefox axi4_blocking_write_read_test/html_cov_report/index.html &
```

Figure 8.11. Coverage report

Then new html window will open

Coverage Summary by Type:

Total Coverage:					19.77%	42.48%
Coverage Type ◀	Bins ◀	Hits ◀	Misses ◀	Weight ◀	% Hit ◀	Coverage ◀
Covergroups	4672	88	4584	1	1.88%	20.93%
Statements	5287	1360	3927	1	25.72%	25.72%
Branches	3567	395	3172	1	11.07%	11.07%
FEC Conditions	20	20	0	1	100.00%	100.00%
Toggles	2336	1278	1058	1	54.70%	54.70%

Figure 8.12. HTML window showing all coverage

Here click on the covergroup there we can see the per instance created and inside that each coverpoint with bins is present there.

Covergroup instance:

\axi4_master_pkg::axi4_master_coverage::axi4_master_covergroup

Summary	Total Bins	Hits	Hit %
Coverpoints	160	35	21.87%
Crosses	2176	12	0.55%

Search:

CoverPoints	Total Bins	Hits	Misses	Hit %	Goal %	Coverage %
ARBURST_CP	4	1	3	25.00%	25.00%	25.00%
ARCACHE_CP	4	2	2	50.00%	50.00%	50.00%
ARID_CP	16	3	13	18.75%	18.75%	18.75%
ARLEN_CP	16	1	15	6.25%	6.25%	6.25%
ARLOCK_CP	2	1	1	50.00%	50.00%	50.00%

Figure 8.13. All coverpoints present in the Covergroup

Coverpoint: ARBURST_CP

Comment:
arburst

Search:

Bin Name	At Least	Hits
ARBURST[READ_FIXED]	1	12
ARBURST[READ_INCR]	1	0
ARBURST[READ_WRAP]	1	0
ARBURST[READ_RESERVED]	1	0

Figure 8.14. Individual Coverpoint Hit

Chapter 9

Test Cases

9.1 Test Flow

In the test, there is virtual sequence and in virtual sequence, sequences are there, sequence_item get started in sequences, sequences will start in virtual sequence and virtual sequence will start in Test

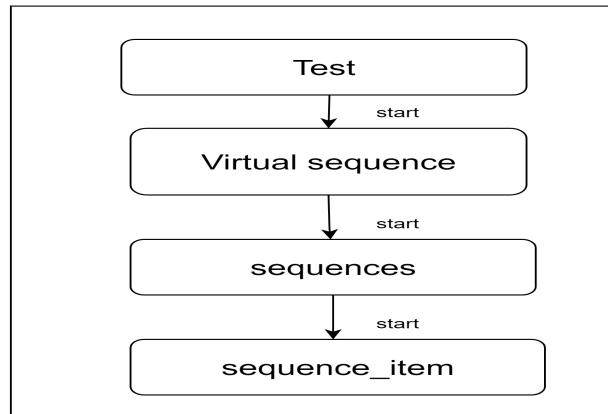


Fig 9.1 Test flow

9.2 AXI4 Test Cases Flow Chart

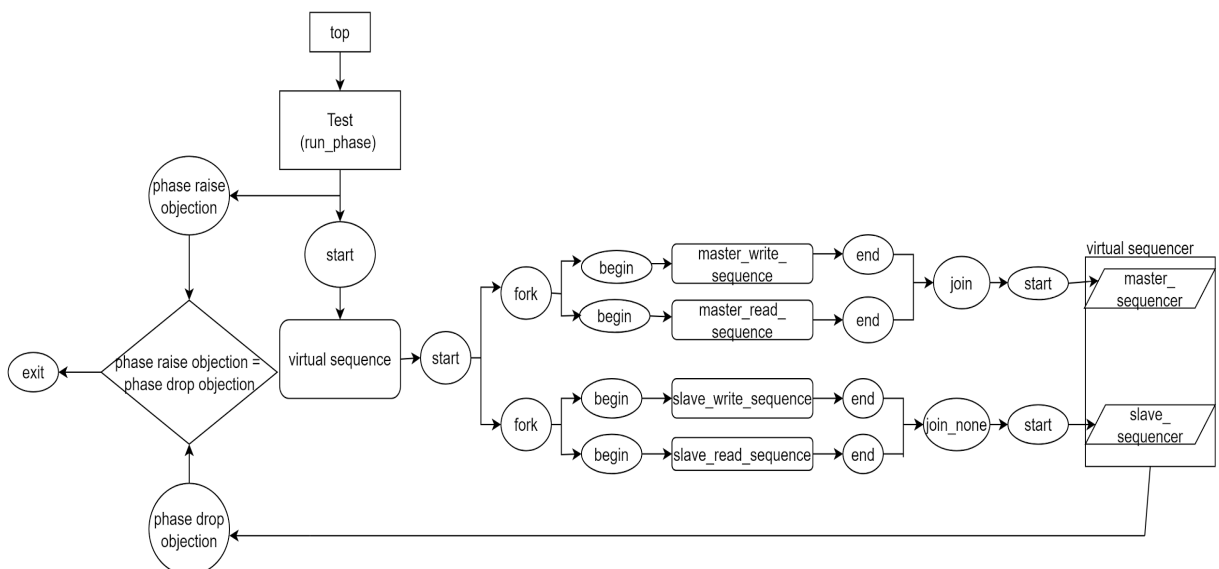


Fig 9.2 AXI4 test cases flow chart

9.3 Transaction

Two types of transactions are there

- Master_tx
- Slave_tx

9.3.1 Master_tx

- Master_tx class is extended from the uvm_sequence_item holds the data items required to drive stimulus to dut
- Declared all the variables(of all the 5 channels, like write address, write data, write response, read address, read data)

Write address channel

- Constraint declared for awburst to get burst type between fixed, incr and wrap types.
- Constraint declared for awlen to restrict the transfer size.
- Constraint declared for awlen to get multiples of 2 in wrap type burst
- Constraint declared for awlock to get locked transfers
- Constraint declared for awburst to select the type of burst
- Constraint declared for awsize to get the size of transfer

```

constraint awburst_c1 { awburst != WRITE_RESERVED;
}
constraint awlength_c2 { if(awburst==WRITE_FIXED || WRITE_WRAP)
    awlen inside {[0:15]};
    else if(awburst == WRITE_INCR)
    awlen inside {[0:255]};
}

constraint awlength_c3 { if(awburst == WRITE_WRAP)
    awlen + 1 inside {2,4,8,16};
}

constraint awlock_c4 { soft awlock == WRITE_NORMAL_ACCESS;
}

constraint awburst_c5 { soft awburst == WRITE_INCR;
}

constraint awsize_c6 { soft awsize inside {[0:2]};
}

```

Fig 9.3 Constraint for write address

Write data channel

- Constraint declared for wdata to restrict data based on awlen
- Constraint declared for no of wait states to restrict the wait states for response

```

constraint wdata_c1 { wdata.size() == awlen + 1;}

constraint no_of_wait_states_c3 { no_of_wait_states inside {[0:3]};}

```

Fig 9.4 Constraint for write data

Read address channels

- Constraint declared for arburst for restricting burst type between incr, wrap and fixed
- Constraint declared for arlen to restrict the transfer size.
- Constraint declared for arlen to get multiples of 2 in wrap type burst

- Constraint declared for arlock to get locked transfers
- Constraint declared for arburst to select the type of burst
- Constraint declared for arsize to get the size of transfer

```

constraint arburst_c1 { arburst != READ_RESERVED;
}
constraint arlength_c2 { if(arburst==READ_FIXED || READ_WRAP)
    arlen inside {[0:15]};
    else if(arburst == READ_INCR)
    arlen inside {[0:255]};
}
constraint arlength_c3 { if(arburst == READ_WRAP)
    arlen + 1 inside {2,4,8,16};
}
constraint arlock_c4 { soft arlock == READ_NORMAL_ACCESS;
}
constraint arburst_c5 { soft arburst == READ_INCR;
}

```

Fig 9.5 Constraint for read address

Memory constraint

- Adding constraint for selecting the endianness

```

constraint endian_c1 { soft endian == LITTLE_ENDIAN;
}

```

Fig 9.6 Constraint for memory

9.3.2 Slave_tx

- Slave_tx class is extended from the uvm_sequence_item holds the data items required to drive stimulus to dut
- Declared all the variables(of all the 5 channels, like write address, write data, write response, read address, read data)
- Constraint declared for ardata to restrict the data based on the arlen

- Constraint declared for the bresp to select the type of write response
- Constraint declared for the rresp to select the type of write response
- Constraint to randomise the wait state in between 0 to 3

```

constraint rdata_c1 { rdata.size() == arlen+1;
                      rdata.size() != 0;
                      }
constraint bresp_c1 {soft bresp == WRITE_OKAY;
                      }
constraint rresp_c1 {soft rresp == READ_OKAY;
                      }
constraint wait_states_c1 {soft no_of_wait_states inside {[0:3]};}

```

Fig 9.7 Constraints for read data response and wait states

Table 9.1 Describing constraint in master and slave transactions

Constraint	Description
awburst_c1	Constraint declared for awburst to get burst type between fixed, incr and wrap types.
awlength_c2	Constraint declared for awlen to restrict the transfer size.
awlength_c3	Constraint declared for awlen to get multiples of 2 in wrap type burst
awlock_c4	Constraint declared for awlock to get locked transfers
awburst_c5	Constraint declared for awburst to select the type of burst
awsiz_c6	Constraint declared for awsize to get the size of transfer
wdata_c1	Constraint declared for wdata to restrict data based on awlen
no_of_wait_states_c3	sponse
arburst_c1	Constraint declared for arburst for restricting burst type between incr, wrap and fixed

arlength_c2	Constraint declared for arlen to restrict the transfer size.
arlength_c3	Constraint declared for arlen to get multiples of 2 in wrap type burst
arlock_c4	Constraint declared for arlock to get locked transfers
arburst_c5	Constraint declared for arburst to select the type of burst
endian_c1	Adding constraint for selecting the endianness
rdata_c1	Constraint declared for ardata to restrict the data based on the arlen
bresp_c1	Constraint declared for the bresp to select the type of write response
rresp_c1	Constraint declared for the rresp to select the type of write response
wait_state_c1	Constraint to randomise the wait state in between 0 to 3

Master tx do_copy,do_compare and do_print methods

- Written functions for do_copy, do_compare, do_print methods, \$casting is used to copy the data member values and compare the data member values and by using a printer , printing the data values.

```

function void axi4_master_tx::do_copy(uvm_object rhs);
    axi4_master_tx axi4_master_tx_copy_obj;

    if(!$cast(axi4_master_tx_copy_obj,rhs)) begin
        `uvm_fatal("do_copy","cast of the rhs object failed")
    end
    super.do_copy(rhs);

    //WRITE ADDRESS CHANNEL
    awid      = axi4_master_tx_copy_obj.awid;
    awaddr    = axi4_master_tx_copy_obj.awaddr;
    awlen     = axi4_master_tx_copy_obj.awlen;
    awsize    = axi4_master_tx_copy_obj.awsize;
    awburst   = axi4_master_tx_copy_obj.awburst;
    awlock    = axi4_master_tx_copy_obj.awlock;
    awcache   = axi4_master_tx_copy_obj.awcache;
    awprot    = axi4_master_tx_copy_obj.awprot;
    awqos     = axi4_master_tx_copy_obj.awqos;

```

```

//WRITE DATA CHANNEL
wdata = axi4_master_tx_copy_obj.wdata;
wstrb = axi4_master_tx_copy_obj.wstrb;

//WRITE RESPONSE CHANNEL
bid   = axi4_master_tx_copy_obj.bid;
bresp = axi4_master_tx_copy_obj.bresp;

//READ ADDRESS CHANNEL
arid   = axi4_master_tx_copy_obj.arid;
araddr = axi4_master_tx_copy_obj.araddr;
arlen  = axi4_master_tx_copy_obj.arlen;
arsize = axi4_master_tx_copy_obj.arsize;
arburst = axi4_master_tx_copy_obj.arburst;
arlock = axi4_master_tx_copy_obj.arlock;
arcache = axi4_master_tx_copy_obj.arcache;
arprot = axi4_master_tx_copy_obj.arprot;
arqos  = axi4_master_tx_copy_obj.arqos;

//READ DATA CHANNEL
rid    = axi4_master_tx_copy_obj.rid;
rdata = axi4_master_tx_copy_obj.rdata;
rresp = axi4_master_tx_copy_obj.rresp;

```

Fig 9.8 do_copy method

```

function bit axi4_master_tx::do_compare (uvm_object rhs, uvm_comparer comparer);
    axi4_master_tx axi4_master_tx_compare_obj;

    if(!$cast(axi4_master_tx_compare_obj, rhs)) begin
        `uvm_fatal("FATAL_axi_MASTER_TX_DO_COMPARE_FAILED", "cast of the rhs object failed")
        return 0;
    end

    return super.do_compare(axi4_master_tx_compare_obj, comparer) &&
    //WRITE ADDRESS CHANNEL
    awid   == axi4_master_tx_compare_obj.awid   &&
    awaddr == axi4_master_tx_compare_obj.awaddr &&
    awlen  == axi4_master_tx_compare_obj.awlen  &&
    awsize == axi4_master_tx_compare_obj.awsize &&
    awburst == axi4_master_tx_compare_obj.awburst &&
    awlock == axi4_master_tx_compare_obj.awlock &&
    awcache == axi4_master_tx_compare_obj.awcache &&
    awprot == axi4_master_tx_compare_obj.awprot &&
    awqos  == axi4_master_tx_compare_obj.awqos  &&

    //WRITE DATA CHANNEL

```

```

//WRITE DATA CHANNEL
wdata == axi4_master_tx_compare_obj.wdata &&
wstrb == axi4_master_tx_compare_obj.wstrb &&

//WRITE RESPONSE CHANNEL
bid == axi4_master_tx_compare_obj.bid &&
bresp == axi4_master_tx_compare_obj.bresp &&

//READ ADDRESS CHANNEL
arid == axi4_master_tx_compare_obj.arid &&
araddr == axi4_master_tx_compare_obj.araddr &&
arlen == axi4_master_tx_compare_obj.arlen &&
arsize == axi4_master_tx_compare_obj.arsize &&
arburst == axi4_master_tx_compare_obj.arburst &&
arlock == axi4_master_tx_compare_obj.arlock &&
arcache == axi4_master_tx_compare_obj.arcache &&
arprot == axi4_master_tx_compare_obj.arprot &&
arqos == axi4_master_tx_compare_obj.arqos &&

//READ DATA CHANNEL
rid == axi4_master_tx_compare_obj.rid &&
rdata == axi4_master_tx_compare_obj.rdata &&
rresp == axi4_master_tx_compare_obj.rresp;
endfunction : do_compare

```

Fig 9.9 do_compare method

```

function void axi4_master_tx::do_print(uvm_printer printer);
//super.do_print(printer);
//`uvm_info("-----WRITE_ADDRESS_CHANNEL", "-----")
printer.print_string("tx_type", tx_type.name());
if(tx_type == WRITE) begin
printer.print_string("awid", awid.name());
printer.print_field("awaddr", awaddr, $bits(awaddr), UVM_HEX);
printer.print_field("awlen", awlen, $bits(awlen), UVM_DEC);
printer.print_string("awsize", awsize.name());
printer.print_string("awburst", awburst.name());
printer.print_string("awlock", awlock.name());
printer.print_string("awcache", awcache.name());
printer.print_string("awprot", awprot.name());
printer.print_field("awqos", awqos, $bits(awqos), UVM_HEX);
//`uvm_info("-----WRITE_DATA_CHANNEL", "-----")

foreach(wdata[i])begin
printer.print_field($sformatf("wdata[%0d]", i), wdata[i], $bits(wdata[i]), UVM_HEX);
end
foreach(wstrb[i])begin
// MSHA: printer.print_field($sformatf("wstrb[%0d]", i), wstrb[i], $bits(wstrb[i]), UVM_HEX);
printer.print_field($sformatf("wstrb[%0d]", i), wstrb[i], $bits(wstrb[i]), UVM_DEC);
end
//`uvm_info("-----WRITE_RESPONSE_CHANNEL", "-----")
printer.print_field("no_of_wait_states", no_of_wait_states, $bits(no_of_wait_states), UVM_DEC);
printer.print_string("bid", bid.name());
printer.print_string("bresp", bresp.name());
end

```

```

if(tx_type == READ) begin
  //`uvm_info("-----READ_ADDRESS_CHANNEL", "-----
  printer.print_string("arid",arid.name());
  printer.print_field("araddr",araddr,$bits(araddr),UVM_HEX);
  printer.print_field("arlen",arlen,$bits(arlen),UVM_DEC);
  printer.print_string("arsize",arsize.name());
  printer.print_string("arburst",arburst.name());
  printer.print_string("arlock",arlock.name());
  printer.print_string("arcache",arcache.name());
  printer.print_string("arprot",arprot.name());
  printer.print_field("arqos",arqos,$bits(arqos),UVM_HEX);
  //`uvm_info("-----READ_DATA_CHANNEL", "-----
  foreach(rdata[i])begin
    printer.print_field($sformatf("rdata[%0d]",i),rdata[i],$bits(rdata[i]),UVM_HEX);
  end
  //printer.print_field("rdata",rdata,$bits(rdata),UVM_HEX);
  printer.print_string("rid",rid.name());
  printer.print_string("rresp",rresp.name());
  printer.print_field("no_of_wait_states",no_of_wait_states,$bits(no_of_wait_states),UVM_DEC);
end
  printer.print_string("transfer_type",transfer_type.name());
endfunction : do_print

```

Fig 9.10 do_print method

Slave tx do_copy,do_compare and do_print methods

- Written functions for do_copy, do_compare, do_print methods, \$casting is used to copy the data member values and compare the data member values and by using a printer , printing the data values.

```

function void axi4_slave_tx::do_copy (uvm_object rhs);
    axi4_slave_tx axi_slave_tx_copy_obj;

    if (!$cast(axi_slave_tx_copy_obj,rhs )) begin
        `uvm_fatal("do_copy","cast of the rhs object failed")
    end

    super.do_copy(rhs);
    //WRITE ADDRESS CHANNEL
    awaddr = axi_slave_tx_copy_obj.awaddr;
    awid   = axi_slave_tx_copy_obj.awid;
    awlen  = axi_slave_tx_copy_obj.awlen;
    awsize = axi_slave_tx_copy_obj.awsize;
    awburst = axi_slave_tx_copy_obj.awburst;
    //awready = axi_slave_tx_copy_obj.awready;
    //awvalid = axi_slave_tx_copy_obj.awvalid;
    awlock = axi_slave_tx_copy_obj.awlock;
    awcache = axi_slave_tx_copy_obj.awcache;
    awqos  = axi_slave_tx_copy_obj.awqos;
    awprot = axi_slave_tx_copy_obj.awprot;

    //WRITE DATA CHANNEL
    wdata = axi_slave_tx_copy_obj.wdata;
    wstrb = axi_slave_tx_copy_obj.wstrb;
    //WRITE RESPONSE CHANNEL
    bid   = axi_slave_tx_copy_obj.bid;
    bresp = axi_slave_tx_copy_obj.bresp;
    //READ ADDRESS CHANNEL

```

```

    //READ ADDRESS CHANNEL
    araddr = axi_slave_tx_copy_obj.araddr;
    arid   = axi_slave_tx_copy_obj.arid;
    arlen  = axi_slave_tx_copy_obj.arlen;
    arsize = axi_slave_tx_copy_obj.arsize;
    arburst = axi_slave_tx_copy_obj.arburst;
    arlock = axi_slave_tx_copy_obj.arlock;
    arcache = axi_slave_tx_copy_obj.arcache;
    arqos  = axi_slave_tx_copy_obj.arqos;
    arprot = axi_slave_tx_copy_obj.arprot;

    //READ DATA CHANNEL
    rid   = axi_slave_tx_copy_obj.rid;
    rdata = axi_slave_tx_copy_obj.rdata;
    rresp = axi_slave_tx_copy_obj.rresp;
    //rready = axi_slave_tx_copy_obj.rready;
    //rvalid = axi_slave_tx_copy_obj.rvalid;
endfunction : do_copy

```

Fig 9.11: slave_tx do_copy method

```

function bit axi4_slave_tx::do_compare (uvm_object rhs, uvm_comparer comparer);
    axi4_slave_tx axi_slave_tx_compare_obj;

    if(!$cast(axi_slave_tx_compare_obj,rhs)) begin
        `uvm_fatal("FATAL_AXI_SLAVE_TX_DO_COMPARE_FAILED","cast of the rhs object failed")
        return 0;
    end

    return super.do_compare(axi_slave_tx_compare_obj, comparer) &&
    //WRITE ADDRESS CHANNEL
    awaddr == axi_slave_tx_compare_obj.awaddr &&
    awid == axi_slave_tx_compare_obj.awid &&
    awlen == axi_slave_tx_compare_obj.awlen &&
    awsize == axi_slave_tx_compare_obj.awsize &&
    awburst == axi_slave_tx_compare_obj.awburst &&
    awlock == axi_slave_tx_compare_obj.awlock &&
    awcache == axi_slave_tx_compare_obj.awcache &&
    awqos == axi_slave_tx_compare_obj.awqos &&
    awprot == axi_slave_tx_compare_obj.awprot &&

    //WRITE DATA CHANNEL
    wdata == axi_slave_tx_compare_obj.wdata &&
    wstrb == axi_slave_tx_compare_obj.wstrb &&

    //WRITE RESPONSE CHANNEL
    bid == axi_slave_tx_compare_obj.bid &&
    bresp == axi_slave_tx_compare_obj.bresp &&

```

```

//READ ADDRESS CHANNEL
araddr == axi_slave_tx_compare_obj.araddr &&
arid == axi_slave_tx_compare_obj.arid &&
arlen == axi_slave_tx_compare_obj.arlen &&
arsize == axi_slave_tx_compare_obj.arsize &&
arburst == axi_slave_tx_compare_obj.arburst &&
//arready == axi_slave_tx_compare_obj.arready &&
//arvalid == axi_slave_tx_compare_obj.arvalid &&
arlock == axi_slave_tx_compare_obj.arlock &&
arcache == axi_slave_tx_compare_obj.arcache &&
arqos == axi_slave_tx_compare_obj.arqos &&
arprot == axi_slave_tx_compare_obj.arprot &&

//READ DATA CHANNEL
rid == axi_slave_tx_compare_obj.rid &&
rdata == axi_slave_tx_compare_obj.rdata &&
rresp == axi_slave_tx_compare_obj.rresp;
//rready == axi_slave_tx_compare_obj.rready &&
//rvalid == axi_slave_tx_compare_obj.rvalid ;
endfunction : do_compare

```

Fig 9.12 slave_tx do_compare method


```

function void axi4_slave_tx::do_print(uvm_printer printer);
//super.do_print(printer);
if(tx_type == WRITE)begin
    //`uvm_info("-----WRITE_ADDRESS_CHANNEL", "-----")
    printer.print_string("awid",awid.name());
    printer.print_field("awaddr",awaddr,$bits(awaddr),UVM_HEX);
    printer.print_field("awlen",awlen,$bits(awlen),UVM_DEC);
    printer.print_string("awsize",awsize.name());
    printer.print_string("awburst",awburst.name());
    printer.print_string("awlock",awlock.name());
    printer.print_string("awcache",awcache.name());
    printer.print_string("awprot",awprot.name());
    printer.print_field("awqos",awqos,$bits(awqos),UVM_HEX);
    //`uvm_info("-----WRITE_DATA_CHANNEL", "-----")
    foreach(wdata[i])begin
        printer.print_field($sformatf("wdata[%0d]",i),wdata[i],$bits(wdata[i]),UVM_HEX);
    end
    foreach(wstrb[i])begin
        printer.print_field($sformatf("wstrb[%0d]",i),wstrb[i],$bits(wstrb[i]),UVM_HEX);
    end
    printer.print_field("wlast",wlast,$bits(wlast),UVM_DEC);
    //`uvm_info("-----WRITE_RESPONSE_CHANNEL", "-----")
    printer.print_string("bid",bid.name());
    printer.print_string("bresp",bresp.name());
end

else if(tx_type == READ) begin
    //`uvm_info("-----READ_ADDRESS_CHANNEL", "-----")
    printer.print_string("arid",arid.name());
    printer.print_field("araddr",araddr,$bits(araddr),UVM_HEX);
    printer.print_field("arlen",arlen,$bits(arlen),UVM_DEC);
    printer.print_string("arsize",arsize.name());
    printer.print_string("arburst",arburst.name());
    printer.print_string("arlock",arlock.name());
    printer.print_string("arcache",arcache.name());
    printer.print_string("arprot",arprot.name());
    printer.print_field("arqos",arqos,$bits(arqos),UVM_HEX);
    //`uvm_info("-----READ_DATA_CHANNEL", "-----")
    foreach(rdata[i])begin
        printer.print_field($sformatf("rdata[%0d]",i),rdata[i],$bits(rdata[i]),UVM_HEX);
    end
    printer.print_string("rresp",rresp.name());

    printer.print_field("no_of_wait_states",no_of_wait_states,$bits(no_of_wait_states),UVM_HEX);
    printer.print_string("TRANSFER_TYPE",transfer_type.name());
end
endfunction : do_print

```

Fig 9.13 slave_tx do_print method

9.4 Sequences

The sequence layer has been simplified. The earlier flat library of blocking (`bk`) and non-blocking (`nbk`) master and slave sequences — one class per data width, burst type and response type — has been retired. Master-side stimulus is now anchored on a single reusable base sequence, `axi4_master_base_seq` (in `sequences/master_sequences/`, exported through `axi4_master_seq_pkg`), and the slave side keeps an analogous base sequence in `sequences/slave_sequences/`. Per-transfer characteristics (width, burst, length, response, outstanding depth) are randomised from the transaction constraints and orchestrated by the mode-specific virtual sequences, rather than by a dedicated sequence class per scenario. This keeps the sequence count small while preserving full coverage of the protocol space.

9.5 Virtual sequences

Virtual sequences coordinate the master and slave sequencers for a given operating mode. They are organised per mode under `test/virtual_sequences/`, each with its own package, on top of the common `axi4_vseq_base_pkg` / `axi4_virtual_base_seq`:

Standalone Master — `virtual_sequences/standalone_master/`

- `axi4_standalone_master_vseq_pkg.sv` (package)
- `axi4_virtual_slave_read.sv`
- `axi4_virtual_slave_write.sv`
- `axi4_virtual_slave_write_read.sv`

Standalone Slave — `virtual_sequences/standalone_slave/`

- `axi4_standalone_slave_vseq_pkg.sv` (package)
- `axi4_virtual_read_seq.sv`
- `axi4_virtual_write_followed_by_read_seq.sv`
- `axi4_virtual_write_read_seq.sv`
- `axi4_virtual_write_seq.sv`

Back-to-Back — `virtual_sequences/back_to_back/`

- `axi4_back_to_back_vseq_pkg.sv` (package)
- `axi4_virtual_back_to_back_read_seq.sv`
- `axi4_virtual_back_to_back_write_read_seq.sv`
- `axi4_virtual_back_to_back_write_seq.sv`

9.6 Test Cases

All tests extend a common base, `axi4_base_test`, provided by `axi4_test_base_pkg`. Above the base, tests are grouped into the three test-package families. Exactly one family is compiled at a time, selected with the `MODE` switch (see §9.7).

Test package family	Package file	Files	Test classes	DUT / role
Standalone Master	<code>axi4_standalone_master_test_pkg</code>	7	6	Master RTL as DUT; master drives, slave passive
Standalone Slave	<code>axi4_standalone_slave_test_pkg</code>	82	81	Slave RTL as DUT; slave responds, master passive
Back-to-Back	<code>axi4_back_to_back_test_pkg</code>	72	71	Active master ↔ active slave VIP

Table 9.6 Test-package families

Representative test names follow the pattern `axi4[_back_to_back]_<outstanding|non_outstanding>_<width>_<burst>_<write|read|write_read>_test`, for example `axi4_non_outstanding_32b_write_read_test` (slave), `axi4_outstanding_slave_write_read_test` (master) and `axi4_back_to_back_outstanding_128b_wrap_burst_write_read_test` (back-to-back). Outstanding tests now enable out-of-order slave responses, and master read-address selection is gated against in-flight/committed writes (ID-based) so read-after-write ordering is respected.

9.7 Testlists

Each test-package family has its own regression testlist under `src/hvl_top/testlists/`:

Testlist	Mode	Tests
<code>axi4_standalone_master_regression.list</code>	master	6
<code>axi4_standalone_slave_regression.list</code>	slave	74
<code>axi4_back_to_back_regression.list</code>	b2b	68

Table 9.7 Regression testlists

Compile-time mode selection

The compile filelist `sim/axi4_compile.f` tags the mode-specific packages with `//@MODE=` markers. The makefile target selects a family in place:

```
cd axi4_avip/sim/questasim
make compile MODE=<b2b|slave|master>      # default: b2b
make simulate test=<test_name> uvm_verbosity=UVM_HIGH
make regression testlist_name=axi4_back_to_back_regression.list
```

where `b2b` builds the back-to-back package, `slave` the standalone-slave package and `master` the standalone-master package. The base test package and shared components are always compiled; only the selected family's test and virtual-sequence packages are enabled.

Chapter 10

User Guide

The user guide is the document that explains how to run tests on different platforms like Questa sim, cadence, and synopsis and also explains how to view waves, coverage.

[User Guide Link](#)

Chapter 11

References

[Reference Link 1](#)

[Reference Link 2](#)

[Reference Link 3](#)

[Reference Link 4](#)